Mesh Agent System: Architecture, Control, and Memory

Technical Report

Project Owner

August 2026 — Version 2

Abstract

This document describes the architecture of the `hello-world` mesh agent system—a multi-agent coordination framework built around heterogeneous LLM backends, bounded execution controllers, durable project autonomy, and governed long-term memory. The report covers LLM backend dispatch; tool routing through `complete_with_tools`; the plan-and-execute harness and its native multi-turn executor; the v2 autonomous project controller, dossiers, admission budgets, and paced wakes; security hardening of subprocess and credential handling; and the full memory hierarchy. That hierarchy spans raw conversation history, three-tier episodic records, an always-on standing digest, cited entity essays, edit-based curation with durable refusal recovery, and a separate human-governed procedural-memory layer. Configuration lives in `mesh.yaml`; runtime dispatch lives in `mesh/llm.py` and `mesh/router_v2.py`; the two controller paths live in `mesh/harness/loop.py` and the autonomous runtime in `mesh/agent_node.py`; memory and project state live beneath `~/.mesh/`.

Contents

I System Architecture **5**

1 Introduction **6**

2 Getting Started **7**
- 2.1 Prerequisites . . . 7
- 2.2 Installation . . . 7
- 2.3 Minimal Configuration . . . 7
- 2.4 Quick Start . . . 8
- 2.5 Verifying It Works . . . 9
- 2.6 Adding More Agents . . . 9
- 2.7 Notes for Public Release . . . 9

3 System Overview **11**

4 LLM Backend Dispatch **13**
- 4.1 Backend Types . . . 13
- 4.2 Live Backend Examples . . . 13
- 4.3 Configuration Reference . . . 13
  - 4.3.1 Backend Configuration (`mesh.yaml`) . . . 13
  - 4.3.2 Agent Node Configuration . . . 13
  - 4.3.3 Specialized LLM Clients . . . 14

5 Tool System **16**
- 5.1 Tool Routing: `complete_with_tools` . . . 16
  - 5.1.1 Dispatch Paths . . . 16
  - 5.1.2 Tool Format Conversion . . . 16
- 5.2 `mesh-tool` CLI . . . 17
- 5.3 MCP Bridge (Legacy) . . . 19
- 5.4 Tool Surface . . . 20
  - 5.4.1 Mesh Tools . . . 20
  - 5.4.2 Harness Tools . . . 20
- 5.5 Backend–Tool Compatibility Matrix . . . 20

II Execution and Control **22**

6 The Mesh Harness **23**
- 6.1 Overview . . . 23

6.2 Harness Toolset Configuration . . . 23
6.3 Tool Sets . . . 24
6.4 Budget and Limits . . . 24
6.4.1 Workspace Calculation . . . 24
6.4.2 Effort-to-Budget Mapping . . . 24
6.5 Configuration Knobs . . . 25

7 Plan-and-Execute Controller **27**
7.1 Phase Flow . . . 27
7.1.1 Phase Tool Exposure . . . 28
7.2 Assessor Controller . . . 28
7.2.1 Controller Invocation . . . 29
7.2.2 Controller Template . . . 29
7.2.3 Evaluator Context . . . 29
7.3 Triage . . . 30
7.4 Observe Phase . . . 30
7.5 Plan Phase . . . 30
7.6 Execute Phase . . . 30
7.6.1 Executor Framing . . . 31
7.7 Observe Cap . . . 31
7.7.1 Layer 1: Template Swap . . . 31
7.7.2 Layer 2: Hard Enforcement . . . 31
7.8 Safety Properties . . . 31
7.9 Key Commits . . . 32

8 Native Multi-Turn Executor Loop **33**
8.1 Location . . . 33
8.2 Activation . . . 33
8.3 Message Format . . . 33
8.4 Cross-Phase Carry-Forward . . . 34
8.5 Per-Iteration Budget Injection . . . 34
8.6 Assessor Boundary . . . 34
8.7 Compaction in Native Mode . . . 35
8.8 `complete_multi_turn` Method . . . 35
8.9 Protocol Events . . . 35

9 Autonomous Project Controller **36**
9.1 Enrollment and Trusted Session Scope . . . 36
9.2 Durable Project Dossier . . . 36
9.3 One Bounded Session . . . 37
9.4 Immutable Reports and Audit Chain . . . 37
9.5 Admission Budgets and Single-Writer Boundaries . . . 37
9.6 Active Mode and Scheduled Wakes . . . 38
9.7 Operator Surface . . . 38

10 Context Management **39**
10.1 Forced Synthesis . . . 39
10.2 Compaction . . . 39
10.2.1 When It Fires . . . 39
10.2.2 Protected Zone . . . 39
10.2.3 Compaction Procedure . . . 40
10.3 Degenerate Loop Detection . . . 40
10.3.1 Tracker State . . . 40
10.3.2 Checkpoint Triggers . . . 40
10.3.3 Three-Level Escalation . . . 40
10.4 CC Watchdog . . . 41
10.4.1 Architecture . . . 41
10.4.2 Evaluation . . . 41
10.4.3 Kill Logic . . . 41
10.5 In-Flight Context Pruning . . . 41
10.5.1 Extreme Result Truncation . . . 41
10.5.2 Tool Result Size Gating . . . 42
10.6 Codex Assessor . . . 42
10.7 Known Discrepancies and Issues . . . 43

III Memory and Knowledge **44**

11 Memory System **45**
11.1 Architecture Overview . . . 45
11.2 Formation V3 Pipeline . . . 46
11.3 Edit-Based Self-Curation . . . 47
11.4 Entity Registry and Cited Essays . . . 48
11.5 Ceilings, Durable Carry-Forward, and Write Audit . . . 48
11.6 Retrieval . . . 49
11.7 Feature Flag Matrix . . . 50
11.8 Standing Digest (Curation-Maintained) . . . 50
11.9 Governed Procedural Memory . . . 51
11.10Key Files . . . 51

12 TOC Selection and Optional Project Maps **53**
12.1 Compatibility Status of Project Maps . . . 53
12.2 Relevance-Based Map Injection . . . 53
12.3 TOC Selection and Weighting (FLMI) . . . 54

IV Infrastructure and Operations **56**

13 Multi-Host Deployment **57**

14 Security Hardening **58**
14.1 Subprocess Environment Allowlist . . . 58
14.2 API Key Masking in Logs . . . 58
14.3 Socket Path Hardening . . . 59
14.4 Node ID Propagation . . . 59
14.5 Agent Node Refactoring . . . 59
14.6 Confirmation Gate Bypass . . . 59
14.7 IDLE→BUSY Race Fix . . . 59

15 Benchmark Infrastructure **60**

V Evaluation **61**

16 Benchmark Results and Ablations **62**

17 Lessons Learned **63**

Part I

System Architecture

Chapter 1

Introduction

The mesh system is a process-based multi-agent coordination framework where users and AI agents communicate through a central message router. Unlike monolithic agent architectures that pair a single LLM with a fixed tool set, the mesh is designed around four principles:

1. **Heterogeneous backends.** Each agent can use a different LLM backend—OpenAI, Anthropic, Claude Code, Z.ai, Codex, or a standalone harness subprocess—selected per-agent via configuration. This enables cost/capability tradeoffs: a cheap local model for routine tasks, an expensive API model for hard problems, and a dedicated assessor LLM for orchestration.

2. **Shared tool surface.** All agents access the same tools—email, calendar, notes, web search, literature, memory, finance, and inter-agent communication—through a unified `mesh-tool` CLI. Tool routing adapts to each backend's capabilities: native function calling for OpenAI/Anthropic, shell invocation for Claude Code/Codex, and structured XML for legacy paths.

3. **Persistent, curated memory.** Agents form broad-recall episodic records from durable conversation history, then maintain a standing digest and cited entity essays through a bounded edit-based curation turn. Semantic and lexical retrieval remain available underneath those curated narratives. Governed skill cards hold procedures in a separate, human-approved lifecycle.

4. **Bounded project autonomy.** An enrolled controller advances one configured project at a time from a durable seven-section dossier. The router meters worker attempts through a persistent daily budget; immutable reports, verified task transitions, and paced one-shot wakes make autonomous progress auditable and restart-resilient.

The system has been in continuous operation since January 2026, running 9 agents across multiple hosts. This report documents the architecture as implemented, including the plan-and-execute controller that orchestrates multi-phase code editing tasks, the native multi-turn executor loop that eliminates redundant file reads across phase boundaries, the autonomous project controller, the v2 memory-curation hierarchy, and the benchmark infrastructure used to evaluate configurations.

Chapter 2

Getting Started

This chapter walks through setting up a minimal mesh instance from scratch: one router, one agent, and a terminal interface.

2.1 Prerequisites

- **Python 3.11+** with `pip` and `venv`.
- An **OpenAI-compatible API key** (for the default backend). For Claude Code backends, a Claude Max subscription and the `claude` CLI binary are required instead.
- **Linux** is the primary development platform. macOS works for the core system; Windows is untested.
- **Git** for cloning the repository.

2.2 Installation

```
git clone <repo-url> hello-world
cd hello-world
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

The `mesh-tool` CLI lives at the repository root and is automatically added to `PATH` via agent prompt injection. For manual use, run it directly:

```
./mesh-tool                    # list all available tools
./mesh-tool gmail_list_recent  # show usage for a specific tool
```

2.3 Minimal Configuration

All configuration lives in `mesh.yaml` at the repository root. The production deployment defines 46 backends and 9 agents, but a minimal setup needs only the router section and one backend.

Environment variables. Create an `env.bash` file exporting the API keys referenced by `mesh.yaml`. At minimum:

```
# env.bash -- DO NOT commit this file
export OPENAI_API_KEY="<your-api-key>"
export MESH_AUTH_TOKEN="$(openssl␣rand␣-hex␣32)"
```

The `MESH_AUTH_TOKEN` is a shared secret used by agents to authenticate with the router. For a local-only setup, any random hex string works.

Minimal `mesh.yaml`. A working single-backend configuration:

```
router:
  host: 127.0.0.1
  port: 7700
  ws_port: 8765
  storage_path: ~/log/chats/mesh-storage/messages.db
  auth_enabled: true
  auth_mode: per_user

llm_backends:
  default:
    backend_type: openai
    api_key: ${OPENAI_API_KEY}
    base_url: https://api.openai.com/v1
    default_model: gpt-4o
    max_tokens: 4096
    temperature: 0.7

nodes:
  agent:researcher:
    llm_backend: default
    system_prompt_file: researcher
```

The `backend_type` field determines which dispatch method is used (see Chapter 4). The `nodes` section maps agent identities to backends and prompt files.

2.4 Quick Start

Three terminal windows are needed: one for the router, one for an agent, and one for the user interface.

Terminal 1 — Router. The router is the central message broker. It classifies incoming messages, manages agent connections, and persists conversation history.

```
source env.bash
python run_router.py
```

Terminal 2 — Agent. Each agent connects to the router via TCP, registers its identity, and waits for dispatched work. The `--agent` flag selects the agent type (which determines the system prompt); `--nickname` sets the display name.

```
source env.bash
python run_agent.py --agent researcher --nickname alice \
    --auth-token $MESH_AUTH_TOKEN
```

Terminal 3 — User TUI. The terminal user interface connects to the router, displays a unified timeline of messages from all agents, and supports target selection, markdown rendering, and session persistence.

```
source env.bash
python run_user_tui.py --nickname yourname
```

On startup the router will prompt for user creation if `auth_mode: per_user` is set. The TUI supports `/target <name>` to set a default recipient and `@<nickname>` mentions to address specific agents.

2.5 Verifying It Works

1. The router terminal should show `[INFO] Router started on 127.0.0.1:7700`.
2. The agent terminal should show `[INFO] Registered as agent:researcher:alice`.
3. The TUI should display a connected status and any presence notifications (e.g., "alice joined").
4. Type a message in the TUI. The router classifies it and dispatches it to alice, who responds via the configured LLM backend.

For a dashboard view of all connected agents, run:

```
mesh-tool mesh_status
```

2.6 Adding More Agents

To add a second agent, start another `run_agent.py` process with a different `--agent` type and `--nickname`:

```
source env.bash
python run_agent.py --agent sysadmin --nickname bob \
    --auth-token $MESH_AUTH_TOKEN
```

Multiple agents can share the same backend or use different ones. To configure per-agent backends, add entries to the `nodes` section of `mesh.yaml` keyed by the full node ID (e.g., `agent:sysadmin:bob`). See Chapter 4 for backend configuration details.

2.7 Notes for Public Release

The production `mesh.yaml` contains 46 backends with paths hardcoded to absolute paths and API keys referenced via environment variables. To prepare the repository for public release:

- Create `env.bash.example` listing required variable names without values.

- Create `mesh.yaml.example` with the minimal configuration shown above (one backend, no hardcoded paths).
- Add `.gitignore` entries for `env.bash`, `firebase-credentials.json`, and any token files.
- Replace hardcoded absolute paths with `$HOME` expansion or relative paths.
- Write a `QUICKSTART.md` linking to this chapter for users who prefer plain-text instructions over the PDF.

Chapter 3

System Overview

The mesh is a multi-agent system where a central **router** classifies user messages and either responds directly or dispatches work to agent nodes. The router has its own LLM backend (`router_v2_llm_backend`) and runs a state machine (IDLE / BUSY) that is distinct from any worker LLM.

Each agent node has:

- An **LLM backend** (how it calls language models),
- A **tool surface** (what tools are available),
- A **persistent memory context** (digest, essays, and episodic retrieval),
- An optional **harness layer** (plan-and-execute controller),
- Optional enrollment as an **autonomous project controller** for an explicit list of `project:*` keys.

Version 2 adds two durable control planes around this message path. The memory plane turns successful formation batches into internal curation turns that maintain the digest and entity essays without holding the message-processing lock. The autonomous plane accepts only runtime-stamped project wakes, carries a compact `SESSION PLAN` across worker-report continuations, meters dispatch attempts in the router, and commits state to project dossiers and immutable session reports.

Router states. RouterV2 exposes three observable states:

IDLE Incoming message → LLM classification call → {`needs_response, needs_worker`}. If `needs_worker` is false, the router generates a direct response and stays IDLE. If true, the router sends an immediate acknowledgment, injects relevant memories, and spawns a worker.

BUSY Worker is running. New messages are handled by the router's own LLM, which peeks at the worker's in-progress context to generate contextual interim responses.

AUTO An autonomous controller session or one of its scoped worker slots is active. The state is derived from trusted session metadata; ordinary interactive workers remain BUSY.

Unified harness-backend dispatch. When the router's own LLM backend is itself a harness-type backend (Claude Code, Codex, or `mesh-harness`—enumerated in `HARNESS_BACKENDS` in `router_v2.py`), the router cannot use native tool calls to launch workers. Instead, these backends share a unified dispatch contract: router-only tools (`worker_launch`, `worker_status`) are stripped from the effective tool catalog, the router receives exactly one outer iteration, and the LLM must emit a `<dispatch_worker>` XML block in its text response to trigger worker creation. Non-harness backends (e.g., DeepSeek direct API) retain the native tool-call dispatch path unchanged.

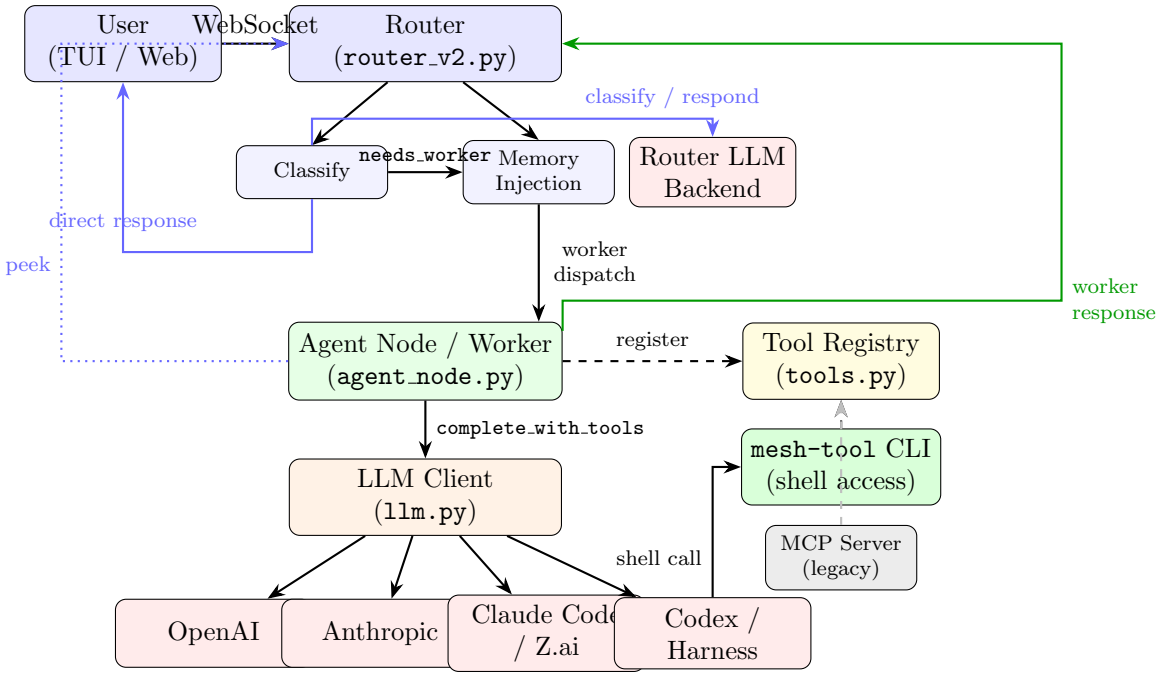

Figure 3.1: Message flow through the mesh. The router has its own LLM backend (`router_v2_llm_backend`, separate from the worker's) used for classification and direct response generation (blue arrows). On `needs_worker`, the router injects relevant memories and dispatches to the agent node, which calls its own LLM backend via `complete_with_tools`. While a worker is running (BUSY), the router generates interim responses via worker peek (dotted blue line). When the worker completes, the agent node sends the response back through the router to the user (green arrow, right side). Codex / Harness subprocesses access mesh tools via the `mesh-tool` CLI. Dashed lines indicate tool discovery; gray indicates legacy paths.

Chapter 4

LLM Backend Dispatch

Backends are configured in `mesh.yaml` under `llm_backends:` and assigned to agents via the `llm_backend` field on each node. The `LLMClient` class in `mesh/llm.py` dispatches to a backend-specific completion method based on `LLMConfig.backend`.

4.1 Backend Types

4.2 Live Backend Examples

The production `mesh.yaml` defines 46 backend configurations. Representative examples:

4.3 Configuration Reference

4.3.1 Backend Configuration (`mesh.yaml`)

Each backend is defined under `llm_backends:` with at minimum:

```
llm_backends:
  my-backend:
    backend_type: openai        # openai | anthropic | claude-code
                                # | zai | codex | mesh-harness
    api_key: ${ENV_VAR}
    base_url: https://...       # optional override
    default_model: gpt-5.1
    max_tokens: 16384
    temperature: 0.7
```

4.3.2 Agent Node Configuration

Each agent node under `nodes:` references a backend and specifies tool access:

```
nodes:
  agent:sysadmin:bob:
    nickname: bob
    llm_backend: claude-code-opus
    system_prompt_file: sysadmin.md
    tools: *common_tools           # YAML anchor
```

Type	Method	Notes
openai	_complete_openai / _complete_openai_responses	Two paths: Chat Completions API for standard models; Responses API for reasoning models (detected via should_use_responses_api). Supports native function calling.
anthropic	_complete_anthropic	Claude API with native tool_use blocks. Supports vision (image extraction from history).
claude-code	_complete_claude_code	Claude Code CLI subprocess. Supports MCP sidecar for mesh tools. Multi-account fallback via cc_fallback_homes.
zai	_complete_zai	Z.ai proxy. Same interface as claude-code (MCP sidecar, CC CLI subprocess).
codex	_complete_codex	Codex CLI subprocess with its own built-in tool loop (codex:shell, codex:patch, etc.). Mesh tools available via mesh-tool CLI on PATH (repo root prepended to subprocess env).
mesh-harness	_complete_mesh_harness	Standalone TAOR (think-act-observe-reflect) loop as a subprocess. Usable as any agent backend, not only for plan-and-execute. Tool set is configurable via harness_toolset (see Chapter 6).

Table 4.1: LLM backend types and their dispatch methods.

```
    memory_enabled: true
    memory_llm_backend: deepseek-direct
    memory_formation_v3_enabled: true
    memory_retrieval_redesign_enabled: true
```

4.3.3 Specialized LLM Clients

Agents may use multiple LLM backends simultaneously:

- **Worker backend** (llm_backend) — primary LLM for conversation and tool use.
- **Router V2 backend** (router_v2_llm_backend) — separate LLM for the routing/classification layer.
- **Memory formation backend** (memory_llm_backend) — dedicated LLM for the v3 segmenter, decoupled from the worker backend (e.g., deepseek-direct while the worker uses claude-code-opus).
- **Assessor backend** (in plan-and-execute) — the controller/checkpoint LLM, configured per mesh-harness backend definition.

Name	Type	Model	Use Case
`default`	openai	gpt-5.1	General agents
`openai-reasoning-*`	openai	o4-mini / o3	Reasoning tasks
`claude-code-opus`	claude-code	opus	Most agents
`deepseek-direct`	openai	deepseek-v4-pro	Memory formation
`local-qwen36`	openai	local-27b	Local executor (Qwen3.6-27B)
`codex-5.5`	codex	gpt-5.5	Codex agents
`mesh-harness-qwen36-pe`	mesh-harness	Qwen3.6-27B + DS-pro	Plan-and-execute

Table 4.2: Selected backend configurations from production `mesh.yaml`.

Chapter 5

Tool System

5.1 Tool Routing: `complete_with_tools`

The central dispatch method is `LLMClient.complete_with_tools()` in `mesh/llm.py`. It accepts a `ToolRegistry`, a list of enabled tool names, and optional MCP configuration, then routes based on backend type.

5.1.1 Dispatch Paths

Path	Condition	Tool Format	Response Parsing
CC briefing	`cc_system_prompt` & `cc_user_prompt` set, backend ∈ {cc, zai}	None (worker dispatch)	Plain text, no tool calls
OpenAI native	`backend == "openai"` & tools	`get_openai_tools()` or `get_openai_responses_tools()`	`_convert_openai_tool_calls()`
Anthropic native	`backend == "anthropic"` & tools	`get_anthropic_tools()`	`_convert_openai_tool_calls()`
CC/zai + MCP	backend ∈ {cc, zai} & `mcp_config` [**Legacy**]	MCP sidecar (tools internal to CC)	Plain text; CC executes tools internally. Being replaced by `mesh-tool` CLI.
XML fall-through	Codex, harness, CC/zai without MCP [**Legacy**]	`format_tools_prompt()` → XML `<mesh_call>` in prompt	`parse_tool_calls()` regex on response. Emits `DeprecationWarning`.

Table 5.1: Tool routing paths in `complete_with_tools`.

5.1.2 Tool Format Conversion

The `ToolRegistry` (in `mesh/tools.py`) provides format converters for each backend:

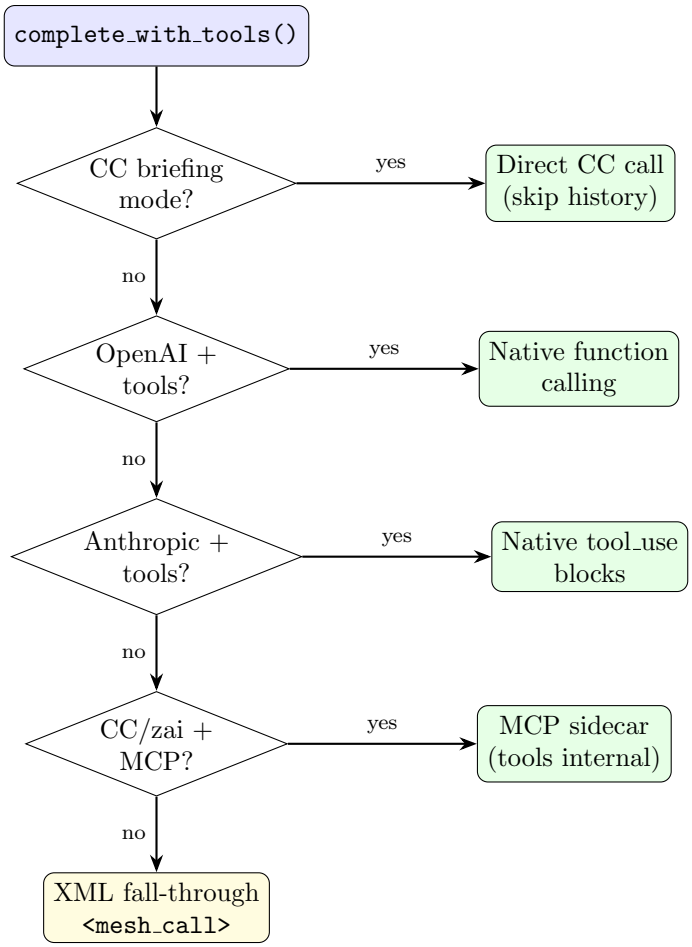

Figure 5.1: Decision tree in `complete_with_tools`. The first matching branch is taken; unmatched backends fall through to XML.

- `get_openai_tools()` — OpenAI Chat Completions function calling format.
- `get_openai_responses_tools()` — OpenAI Responses API flattened tool format.
- `get_anthropic_tools()` — Anthropic `tool_use` format.
- `format_tools_prompt()` — XML `<mesh_call>` prompt block for completion-only backends.

5.2 `mesh-tool` CLI

The `mesh-tool` CLI provides shell-based access to mesh tools, replacing the MCP sidecar for all backends with shell access. Any agent that can execute shell commands can call mesh tools directly—no schema injection, XML parsing, or MCP plumbing required.

Architecture. `mesh-tool` is a bash wrapper at the repository root that activates the virtualenv and runs `python -m mesh.cli.mesh_tool`. It imports the same `tool_implementations.py` functions used by the `ToolRegistry`—no reimplementation.

```
mesh-tool (bash wrapper, repo root)
  -> python -m mesh.cli.mesh_tool
```

```
    -> imports mesh.tool_implementations
    -> looks up tool in ToolRegistry
    -> parses --key value args (type coercion)
    -> executes handler (async-aware)
    -> JSON to stdout, errors to stderr
```

Invocation contract.

- `mesh-tool` (no args) — lists all available tools grouped by category.
- `mesh-tool <name>` (no required args provided) — prints usage with required/optional parameters.
- `mesh-tool <name> --help` — same as above.
- `mesh-tool <name> --arg value ...` — executes the tool.
- Wrong/missing arguments — prints error to stderr with "Did you mean?" suggestions, usage to stdout, exit 1.

Output contract.

- **Success**: JSON to stdout, exit 0.
- **Error**: human-readable message to stderr, usage to stdout, exit 1.
- **Help**: usage text to stdout, exit 0.

Identity. The environment variable `MESH_NODE_ID` identifies the calling agent (e.g., `agent:sysadmin:bob`). It is propagated through two layers: (1) `os.environ` is set at agent construction for tool subprocess inheritance (`bash_exec`, etc.); (2) `LLMConfig.node_id` is threaded explicitly into each LLM subprocess environment (`_run_cc_subprocess`, `_complete_codex`, `_complete_mesh_harness`). The explicit path ensures correctness if multiple agents share a process.

PATH wiring. The agent runtime prepends the repository root to `PATH` in both `_run_cc_subprocess()` and `_complete_codex()`, making `mesh-tool` discoverable by any subprocess spawned by these backends.

Prompt injection. All agent prompts auto-include `mesh/prompts/mesh_tools.md` via the `load_prompt_file()` auto-include mechanism (same pattern as `channel_policy.md` and `memory.md`). This gives agents concise instructions on discovering and calling mesh-tool from the shell.

Tool surface. 53 tools across Gmail, web search, notes, literature, calendar, memory, maps, mesh info, finance, and utility categories. Plus `send_message` for agent communication.

Agent-routed tools. Tools requiring agent-local state (`send_message`, `channel_list`, `mesh_status`, `agent_status`, `schedule_wake`, `gmail_send_message`, `gmail_reply_to`, `account_set_current`, etc.) are dispatched via the agent's Unix socket at `~/.mesh/sockets/{sanitized_id}.sock` (directory mode 0700, owner-only access). The CLI falls back to the legacy `/tmp/mesh_agent_*` path for agents started before the hardening change. Memory tools also prefer the socket path when available.

Gmail account switching. All Gmail tools accept an `--account` flag for stateless per-call account selection (e.g., `mesh-tool gmail_list_recent --account personal --limit 5`). Each CLI invocation is a fresh subprocess; there is no persistent account state between calls. Write tools

(gmail_send_message, gmail_reply_to) are routed through the agent socket with skip_confirmation=True; the socket handler saves the current account, switches to the requested account before executing, and restores in a try/finally block.

Stdin mode (shell interpolation hardening). When a parameter value is passed as - and stdin is not a TTY, mesh-tool reads the value from standard input instead of from the command-line argument. This enables single-quoted heredocs to bypass bash expansion of $, backticks, and $(...) constructs:

```
mesh-tool gmail_send_message --to "x@y.com" --subject "Invoice" --body -
   <<'EOF'
The payment of $550 is still outstanding.
EOF
```

Without this mechanism, $550 would be interpolated as the shell variable $5 concatenated with 50, producing a mangled message. The fix is a three-layer approach: (1) stdin mode in the CLI parser (mesh_tool.py), (2) shell-safety instruction sections in agent prompts documenting the <<'EOF' convention, and (3) the documented convention in mesh/prompts/mesh_tools.md.

Map tool socket routing. The tools map_edit, map_create, map_review, and set_project_context are routed through the agent socket via a prefix-matching check in _execute() (not via AGENT_ROUTED_TOOLS, which is reserved for placeholder-handler tools). This ensures that worker subprocesses can modify the optional legacy project-map subsystem via the parent agent's initialized memory system rather than attempting to initialize their own (which would fail with "memory not initialized" errors).

gmail_list_recent / gmail_list_unread. Two convenience tools for inbox browsing: gmail_list_recent returns the N most recent emails (newest first, no query required), and gmail_list_unread returns unread emails only. Both accept the --account flag and are primarily used by assistant agents for unknown-target email retrieval.

5.3 MCP Bridge (Legacy)

Status: Being replaced by mesh-tool CLI (§5.2). The MCP sidecar is still functional but is no longer the recommended path. New agents should use mesh-tool via shell instead.

For claude-code and zai backends, mesh tools were historically exposed via an MCP (Model Context Protocol) server sidecar defined in mesh/mcp_server.py. The CC subprocess is started with --mcp-config pointing to a temporary JSON file that configures the MCP server.

Agent-local tools. A subset of tools require agent-local state (WebSocket connection, scheduler, etc.) and are routed to agent_node.py via Unix socket:

> send_message, channel_list, channel_members, schedule_wake, schedule_list, schedule_cancel, agent_shutdown, mesh_status, agent_status

5.4 Tool Surface

5.4.1 Mesh Tools

Mesh tools are defined in `mesh/tool_implementations.py` and registered via `mesh/clients/account_manager.py` (the `ToolHost`). They cover external services:

Category	Tools
Email	`gmail_list_from_date`, `gmail_get_email`, `gmail_send_message`[S], `gmail_reply_to`[S], `gmail_search_emails`, `gmail_list_recent`, `gmail_list_unread`
Calendar	`calendar_list_on_date`, `calendar_create_event`[C], `calendar_delete_event`[C]
Notes	`notes_search`, `notes_list`, `notes_get`, `notes_read`, `notes_add`, `notes_delete`, `notes_edit`
Web	`exa_search`, `exa_fetch_full`, `extract_url`
Literature	`literature_search`, `literature_fulltext`, `arxiv_{search,get,fulltext}`, `pubmed_{search,get,fulltext,related}`
Memory	`memory_{search,get,list,add,delete}`, `map_{get,list,edit,create,review}`, `set_project_context`, `remember`
Finance	`plaid_{link_start,link_status,accounts,sync, transactions,unlink}`
Mesh	`send_message`, `mesh_list`, `mesh_status`, `agent_status`, `agent_shutdown`, `channel_{list,members}`
Scheduling	`schedule_{wake,list,cancel}`
Browser	`browser_{session_*,goto,get_url,snapshot_controls, read_text,click,fill,type,press,select,back}`
Utility	`current_time`, `tool_help`, `echo`, `account_{get_current,list,set_current}`, `personality_{get,set}`, `synthetic_quota`, `claude_code_usage`

Table 5.2: Mesh tools by category. [S] = socket-routed (via agent Unix socket with `skip_confirmation`). [C] = confirmation-gated (requires user approval).

5.4.2 Harness Tools

The harness layer (`mesh/harness/tools/__init__.py`) defines three tool sets that mirror or extend the Codex tool surface:

5.5 Backend–Tool Compatibility Matrix

Set	Tools	Purpose
HARNESS_TOOLS (8)	apply_patch, shell, list_dir, file_read, file_edit, phase_complete, grep, find_files	Full executor surface. Registered with the global ToolRegistry on import.
PLANNER_TOOLS (4)	file_read, list_dir, grep, find_files	Read-only subset for the observe phase.
MESH_READ_ONLY_TOOLS (45)	All read-only mesh tools (search, list, get variants; no write/confirmation tools)	Safe-to-call during observe phase. Filtered from agent's full tool list.

Table 5.3: Harness tool sets.

Backend	Native	XML	MCP	mesh-tool	Mesh Tools	Harness
openai	✓				✓	via harness
anthropic	✓				✓	via harness
claude-code			†	✓	✓	via harness
zai			†	✓	✓	via harness
codex				✓	✓	built-in
mesh-harness		†		✓	✓	✓

Table 5.4: Backend–tool compatibility. "Native" = API-level function calling. "XML" = <mesh_call> in prompt. "MCP" = tools via MCP sidecar. "mesh-tool" = tools via shell CLI. † = deprecated path (still functional but being phased out). "via harness" = available when the backend runs inside the harness layer. **Key change**: codex now has mesh tool access via mesh-tool CLI.

Part II

Execution and Control

Chapter 6

The Mesh Harness

The mesh harness is a standalone TAOR (Think-Act-Observe-Repeat) execution loop implemented in `mesh/harness/loop.py`. It can operate as a simple iterative executor (standard mode) or as the substrate for the plan-and-execute controller (Chapter 7).

6.1 Overview

All harness logic resides in a single file: `mesh/harness/loop.py`. Supporting definitions are in `mesh/harness/tools/__init__.py` (tool sets), `mesh/harness/protocol.py` (event emission), and `mesh/config.py` (configuration plumbing).

The entry point is `run_loop()`, which accepts an LLM client, conversation history, tool names, and configuration parameters. In standard mode, it runs a simple loop: call the LLM, execute any tool calls, append results to history, repeat until the LLM produces a final response or hits the iteration limit.

6.2 Harness Toolset Configuration

The `mesh-harness` backend controls which tools the subprocess receives via two config fields on `LLMConfig`:

- `harness_toolset` (default: `"legacy"`) — `"legacy"` exposes the full mesh tool set (email, calendar, notes, memory, browser, etc. via the parent agent's `ToolRegistry`); `"harness"` restricts to the 4-tool Codex-style surface (`file_read`, `list_dir`, `grep`, `find_files` plus `apply_patch`, `shell`).
- `harness_tools` (default: `""`) — comma-separated tool names. When non-empty, *overrides* `harness_toolset` entirely, giving fine-grained control.

The toolset flag is passed to the subprocess via `--toolset` or `--tools` CLI arguments (see `_complete_mesh_harness` in `llm.py`).

Codex as direct backend vs. through harness. There is an important distinction between two ways an agent can use Codex:

1. **Direct backend** (`backend_type: codex`) — `_complete_codex` launches the Codex CLI subprocess with its own built-in tools (`codex:shell`, `codex:patch`, etc.). Mesh tools are now available via `mesh-tool` CLI on `PATH` (repo root prepended to subprocess environment).
2. **Through harness** (`backend_type: mesh-harness` with a codex sub-backend) — the harness wraps the Codex model in its own TAOR loop and can expose mesh tools via `harness_toolset:`

`legacy`. In this mode, mesh tools are available to the harness loop but the inner model does not see them directly.

6.3 Tool Sets

Defined in `mesh/harness/tools/__init__.py`:

Set	Tools
`HARNESS_TOOLS`	`apply_patch`, `shell`, `list_dir`, `file_read`, `file_edit`, `phase_complete`, `grep`, `find_files`
`PLANNER_TOOLS`	`file_read`, `list_dir`, `grep`, `find_files`
`MESH_READ_ONLY_TOOLS`	45 tools: `gmail_*`, `exa_*`, `notes_*`, `arxiv_*`, `pubmed_*`, `calendar_*`, `memory_*`, `map_*`, `browser_*`, `plaid_*`, etc.
`READ_ONLY_TOOLS`	`file_read`, `list_dir`, `grep`, `find_files` (used by degenerate escalation Level 1)
`WRITE_TOOLS`	`apply_patch`, `file_edit`, `file_create`, `file_write` (plus shell write pattern detection)

Shell write detection. `_is_write_call()` classifies `shell` calls as writes if their command matches `SHELL_WRITE_PATTERNS`:

```
r"\b(sed\s+-i|tee\s|patch\s|mv\s|rm\s|cp\s|chmod\s|
    chown\s|mkdir\s|touch\s)"
r"|[^|]>(?!&)"    # redirect (not pipe)
r"|>>"             # append
```

6.4 Budget and Limits

†`MAX_ITERATIONS` is 100 in the code constant (`loop.py:45`), but the CLI `--max-iters` flag defaults to 50 (`__main__.py:213`). Agents running without an explicit `--max-iters` argument will hit the 50-iteration limit, not 100.

6.4.1 Workspace Calculation

$$\texttt{workspace} = \max(\texttt{soft\_limit} - \texttt{initial\_tokens},\ 50{,}000)$$
$$\texttt{compaction\_threshold} = \lfloor \texttt{workspace} \times 0.40 \rfloor$$
$$\texttt{observe\_limit} = \min(\lfloor \texttt{workspace} \times 0.50 \rfloor,\ 250{,}000)$$

6.4.2 Effort-to-Budget Mapping

From `mesh/harness/__main__.py`. This maps the `cc_effort` string to `anthropic_thinking_budget` (thinking tokens for extended reasoning), *not* to the context soft limit:
The context `soft_limit` is independently configurable via `harness_soft_limit` (default: 500,000).

Constant	Default
MAX_ITERATIONS	$100^{\dagger}$
DEFAULT_SOFT_LIMIT	500,000 tokens
KEEP_RECENT_RESULTS	4
FORCED_SYNTHESIS_FRACTION	0.97
LLM_RETRIES	4
LLM_RETRY_BACKOFF	$(2, 4, 8, 16)$ seconds
TRANSIENT_HTTP_CODES	$\{400, 429, 500, 502, 503, 504\}$
CHECKPOINT_STALL_TURNS	3
DEGENERATE_ESCALATION_LIMIT	3
DEFAULT_MAX_PHASES	15
DEFAULT_COMPACTION_THRESHOLD_FRACTION	0.40
ASSESSMENT_MAX_RETRIES	2
PLAN_MAX_RETRIES	2
HALLUCINATION_MAX_RETRIES	2
WATCHDOG_MAX_CONTEXT_TOKENS	100,000
CCWatchdog.RECENT_WINDOW	16

Effort	anthropic_thinking_budget (tokens)
low	2,000
medium	5,000
high	10,000
xhigh	50,000

6.5 Configuration Knobs

All P&E-relevant configuration flows through CLI arguments in `mesh/harness/__main__.py` and the `LLMBackendConfig` class in `mesh/config.py`:
Checkpoint tuning knobs (per-call, not config):

Config Field	Type	Default
harness_controller_mode	str	"standard"
harness_soft_limit	int	500,000
harness_max_phases	int	15
harness_compaction_threshold_fraction	float	0.40
harness_backend	str	(varies)
harness_toolset	str	"legacy"
harness_assessor_model	str	(varies)
harness_assessor_backend	str	(varies)
thinking_budget*	int	(none)
harness_assessor_effort	str	(none)
harness_codex_assessor	bool	false
harness_codex_assessor_model	str	(none)
harness_codex_assessor_effort	str	(none)
harness_codex_assessor_binary	str	"~/.local/bin/codex"

Table 6.1: P&E configuration fields on LLMBackendConfig (mesh/config.py). *thinking_budget lives on LLMConfig (mesh/llm.py), not LLMBackendConfig; it is wired through via backend_config_to_llm_config(). In production, harness_assessor_effort: xhigh is set only on mesh-harness-qwen36-codex-pe, not on all P&E backends.

Parameter	run_loop() arg	P&E default
Periodic interval	checkpoint_periodic_interval	5
Periodic start	checkpoint_periodic_start	8
Stall turns	CHECKPOINT_STALL_TURNS	3 (hardcoded)
Escalation limit	DEGENERATE_ESCALATION_LIMIT	3 (hardcoded)

Chapter 7

Plan-and-Execute Controller

The Plan-and-Execute (P&E) controller is a multi-phase execution controller that wraps the standard TAOR loop with an outer assessor controller. At each phase boundary, the assessor chooses one of four intents—`observe`, `plan`, `execute`, or `terminate`.

The controller exists to decompose complex tasks (code edits, bug fixes, document production) into sequential phases, each with a focused objective and bounded context. A separate assessor LLM (typically DeepSeek-v4-pro) drives phase transitions, while the primary LLM (the executor) performs the actual work within each phase.

Entry point. The P&E controller is activated when `run_loop()` is called with `controller_mode="plan_and_execu` It immediately delegates to `_run_plan_and_execute_loop()`, which runs the outer control loop.

7.1 Phase Flow

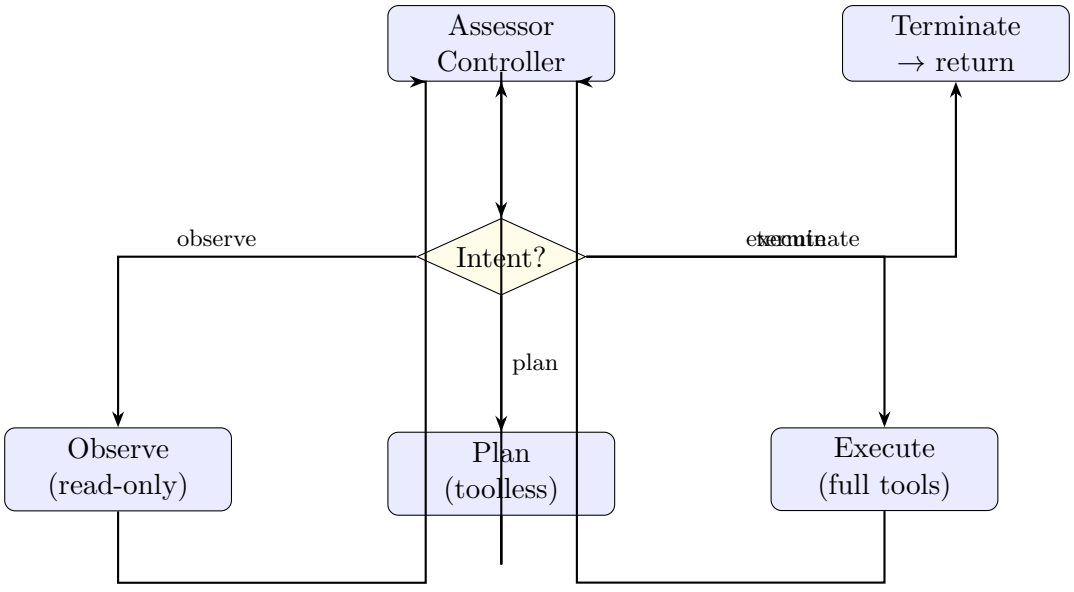

Figure 7.1: P&E controller phase flow. Each non-terminal phase loops back to the assessor controller, which selects the next intent.

The controller iterates up to `DEFAULT_MAX_PHASES = 15`. At each iteration, it:

1. Builds an evaluator context XML document via `_build_evaluator_context()`.
2. Appends an `<action_history>` section summarizing all prior phases.
3. Calls the assessor LLM with the controller template and `max_iterations=1` (no tools, single turn).
4. Parses the response via `parse_controller_intent()` into a `ControllerIntent` dataclass.
5. Dispatches to the appropriate phase handler.

Default on parse failure. If the intent tag cannot be parsed, `parse_controller_intent()` defaults to `observe`—the safest option since it is read-only.
Phase cap enforcement. If the loop exhausts all `max_phases` iterations without terminating, a hard error message is returned.

7.1.1 Phase Tool Exposure

Phase	LLM	Available Tools
Controller (assessor)	Assessor (e.g., DS-pro)	`tool_names=[]` — no tools; emits XML intent only
Observe	Executor backend	`PLANNER_TOOLS` ∪ (agent's `tool_names` ∩ `MESH_READ_ONLY_TOOLS`)
Plan	Controller LLM	`tool_names=[]` — single call, parses plan XML
Execute	Executor backend	Full agent `tool_names` (all configured tools + `HARNESS_TOOLS`)

Table 7.1: Tool exposure per plan-and-execute phase. The observe phase cannot trigger write or confirmation-gated tools.

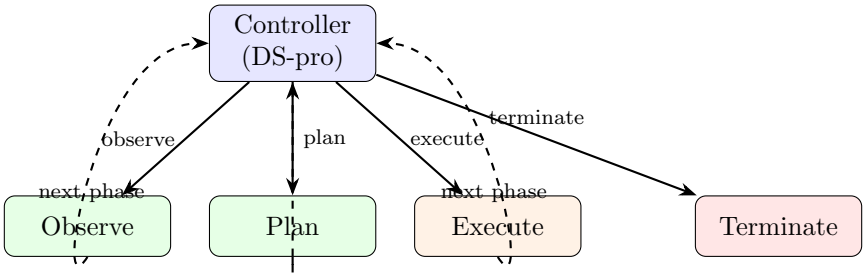

Figure 7.2: Controller loop. Each phase feeds back to the controller for the next intent decision. Terminate exits the loop and returns the final response.

7.2 Assessor Controller

The assessor is a separate LLM client (`assessor_llm_client`), defaulting to the primary `llm_client` if not provided. In production, this is typically DeepSeek-v4-pro.

7.2.1 Controller Invocation

The controller is invoked as a *single-turn, toolless* `run_loop()` call:

```
controller_output = await run_loop(
    llm_client=_assessor,
    history=[HistoryMessage(
        from_node="user",
        content=eval_ctx,     # evaluator context XML
        ...)],
    tool_names=[],            # no tools
    max_iterations=1,         # single turn
    instructions=phase_instructions,  # controller template
    ...
)
```

The controller sees a fresh single-message history containing only the evaluator context—it does not share the executor's conversation history.

7.2.2 Controller Template

The template (`_CONTROLLER_TEMPLATE`) presents four intents with XML output format:

- `<intent>observe</intent>` + `<prompt>`
- `<intent>plan</intent>` + `<plan>`
- `<intent>execute</intent>` + `<prompt>`
- `<intent>terminate</intent>` + `<summary>`

7.2.3 Evaluator Context

The function `_build_evaluator_context()` constructs an XML document:

```
<evaluator_context>
  <task>...</task>
  <plan>...</plan>            <!-- current plan, if any -->
  <prior_phases>              <!-- compacted phase summaries -->
    <phase_summary phase="N">...</phase_summary>
  </prior_phases>
  <current_phase phase="N">
    <objective>...</objective>
    <tool_history>            <!-- faithful tool call/result copies -->
      <tool_call name="...">...</tool_call>
    </tool_history>
    <files_modified>...</files_modified>
  </current_phase>
  <action_history>            <!-- appended by P&E loop -->
    - Phase 1 (observe): ...
    - Phase 2 (plan): ...
  </action_history>
</evaluator_context>
```

Prior phase summaries are populated either from an explicit list or by scanning history messages for `<phase_summary>` tags. Files modified are tracked by matching `WRITE_TOOLS` tool names and

extracting file paths via regex.

7.3 Triage

Note: Triage is *not* currently invoked by the P&E controller loop. `_TRIAGE_TEMPLATE` and `parse_triage()` exist in the codebase but the P&E loop does not call them—the first phase simply goes to the controller for an intent decision. The controller template's guidelines ("If the task is a simple factual question you can answer now, terminate immediately") serve the same purpose implicitly.

Triage classifies tasks into three categories:

1. **Done** — answerable from knowledge, no tools needed.
2. **Search-only** (`needs_plan=false`) — pure information retrieval; findings are the deliverable.
3. **Needs plan** (`needs_plan=true`) — requires producing an artifact (code, documents, etc.).

The `TriageDecision` dataclass captures the parse result. On parse failure, `parse_triage()` defaults to `needs_plan=True`.

7.4 Observe Phase

When the controller selects `observe`, the executor runs a read-only exploration loop:

- **Tools:** `search_tools` = `PLANNER_TOOLS` (`file_read`, `list_dir`, `grep`, `find_files`) plus any `MESH_READ_ONLY_TOOLS` found in the task's `tool_names`.
- **Soft limit:** `observe_limit` = $\min(\lfloor$`workspace` $\times 0.50 \rfloor, 250{,}000)$.
- **Checkpoints:** enabled, with `checkpoint_periodic_interval=5`, `checkpoint_periodic_start=8`.
- **Instructions:** `OBSERVE_INSTRUCTIONS` appended to the controller's prompt.

No compaction after observe. History from observe phases is preserved intact so that subsequent plan/execute phases can see the raw tool results (commit `d8d74ca5`).

Phase discipline is advisory, not enforced. The observe phase uses `PLANNER_TOOLS` (read-only) but this is enforced only at the tool-set level. If the executor's backend (e.g., Claude Code) has its own tools, those are not restricted by the harness tool-name filter.

7.5 Plan Phase

When the controller selects `plan`, the plan is extracted inline from the controller's own output—no separate LLM call is made. The plan is stored in `current_plan` and injected into subsequent executor phases via `_EXECUTOR_FRAMING` and into assessments via `_ASSESSMENT_TEMPLATE`.

Plan revision. The controller can revise the plan at any time by selecting `plan` again. Additionally, the legacy assessment path supports `<revised_plan>` tags that replace `current_plan`.

Relationship to legacy plan phase. The `PLAN_INSTRUCTIONS` template and `extract_plan_and_phase_prompt()` are remnants of the earlier fixed-sequence controller (observe → plan → execute). In the current assessor-controller design, the plan phase is handled inline by the controller itself, and these functions are not called by the P&E loop.

7.6 Execute Phase

When the controller selects `execute`, a full-power TAOR loop runs:

- **Tools:** the original `tool_names` passed to `run_loop()` (typically `HARNESS_TOOLS` plus mesh read/write tools).
- **Soft limit:** the full `soft_limit` (not the reduced observe limit).
- **Instructions:** `_EXECUTOR_FRAMING` template wraps the controller's prompt with plan context.
- **Checkpoints:** enabled, `periodic_interval=5`, `periodic_start=8`.
- **CCWatchdog:** enabled for `claude-code` and `zai` backends when `assessor_llm_client` is provided.

7.6.1 Executor Framing

The executor receives its instructions via `_EXECUTOR_FRAMING`, which injects the current plan and phase-specific goal into a structured template. The framing tells the executor to focus only on the current phase's goal and not to start the next phase.

7.7 Observe Cap

After 2 observe phases (lifetime, not per-segment), the observe intent is disabled. This is a two-layer enforcement (commit `bb62ff3c`):

7.7.1 Layer 1: Template Swap

When `observe_count >= 2`, the controller template is replaced with `_CONTROLLER_TEMPLATE_NO_OBSERVE` via `_strip_observe_from_instructions()`. The no-observe template begins with:

> ***observe is temporarily disabled*** *— you have already used your observation budget. Choose plan, execute, or terminate.*

If the template swap fails (template not found in instructions), a `RuntimeError` is raised as a drift-protection mechanism.

7.7.2 Layer 2: Hard Enforcement

If the assessor LLM ignores the template and returns `observe` anyway, the phase is *skipped* (not force-terminated). A warning is logged and the action history records the skip.
Lifetime cap. The `observe_count` is incremented on each observe phase and is *never reset*—not even after execute phases. The cap is lifetime, not per-segment.

7.8 Safety Properties

- **Observe cap**: After 2 observe phases, the observe intent is permanently removed from the controller's option set for the remainder of the task.
- **Observe checkpoints**: Every 5 iterations starting at iteration 8, the assessor is asked whether to continue or complete the phase.
- **History carryover**: Observe-phase tool results (file reads, searches) are preserved in the shared `history` list for subsequent phases. No compaction fires between observe and execute—only between consecutive execute phases.
- **Read-only observe**: The `MESH_READ_ONLY_TOOLS` whitelist ensures the observe phase cannot send emails, create events, or perform other write operations.

7.9 Key Commits

`54a33f8a`
feat: plan-and-execute controller, codex backend, assessor streaming, greenfield benchmarks. Initial P&E implementation with fixed observe → plan → execute sequence and assessor-driven phase transitions.

`c4bc1763`
feat: flexible assessor controller, synthesis/compaction dead-zone fix, gpt-oss reasoning fallback. Replaced fixed sequence with flexible four-intent controller (observe/plan/execute/terminate).

`bb62ff3c`
fix: P&E controller — lifetime observe cap, skip-not-force semantics, helpers. Changed observe cap from per-segment to lifetime.

`d8d74ca5`
fix: preserve observe-phase history for executor — stop compacting between phases.

`177d8689`
feat(harness): per-turn context budget meter + forced-synthesis cutoff. Added continuous budget awareness line and the forced synthesis mechanism.

Chapter 8

Native Multi-Turn Executor Loop

The XML-serialized history path (`run_loop`) has a fundamental deficiency for multi-phase execution: tool results from prior phases appear to the executor as `<message from="system">` XML text rather than native API tool results. The executor model treats them as someone else's notes and re-reads the same files, duplicating context and accelerating compaction.

The **native executor loop** (`run_native_loop`) solves this by maintaining the conversation as a native OpenAI `messages: list[dict]` instead of `list[HistoryMessage]`. Tool results are stored as `{"role": "tool", "tool_call_id": "..."}` messages—the executor treats them as its own prior work and builds on them across phase boundaries.

8.1 Location

`mesh/harness/loop.py`
: `run_native_loop()`, plus helper functions `estimate_native_tokens()`, `_tool_name_for_call_id()`, `_build_evaluator_context_from_native()`, `_compact_native_history()`.

`mesh/llm.py`
: `complete_multi_turn()`—OpenAI Chat Completions call that accepts pre-built `messages` and skips XML serialization entirely.

`mesh/tools.py`
: `ToolCall.call_id` field—preserves the API-returned tool call ID for native message linking.

8.2 Activation

Native mode activates when `backend == "openai"` inside `_run_plan_and_execute_loop()`. Non-OpenAI backends (CC, zai, mesh-harness XML) fall back to the existing XML path. The dispatch is purely conditional—no configuration flag required.

8.3 Message Format

The conversation uses standard OpenAI Chat Completions format:

```
# System + task (protected zone, never compacted)
{"role": "system", "content": "<system_prompt + tool_prompt>"}
{"role": "user",   "content": "<task + instructions>"}
```

```
# Phase framing (injected by P&E controller before each phase)
{"role": "user",   "content": "<observe/execute instructions>"}

# Assistant turn with tool calls
{"role": "assistant", "tool_calls": [
    {"id": "call_abc", "type": "function",
     "function": {"name": "file_read", "arguments": "..."}}
]}
# Tool result linked by call_id
{"role": "tool", "tool_call_id": "call_abc",
 "content": "<file contents>"}
```

8.4 Cross-Phase Carry-Forward

The `native_messages` list is shared across observe and execute phases via the P&E controller. When the assessor transitions from observe to execute:

1. Observe phase tool results remain in `native_messages` as native `role=tool` messages.
2. The P&E controller appends the executor framing as a `role=user` message.
3. The execute phase calls `run_native_loop()` with the same `native_messages` list.
4. The executor LLM sees all observe-phase file reads as its own prior tool results and does not re-read them.

8.5 Per-Iteration Budget Injection

On each iteration, `run_native_loop` appends a transient `role=user` message containing the budget line and any per-iteration instructions. This message is removed after the LLM responds so it does not accumulate in the conversation history. During forced synthesis, the message stays (it is the final instruction to produce output).

8.6 Assessor Boundary

The assessor remains on the XML path. When the P&E controller needs a phase decision:

1. `_build_evaluator_context_from_native()` converts the native messages into structured XML (`<evaluator_context>`).
2. The assessor call uses `run_loop` (XML mode) with a single-message history containing the evaluator context.
3. The assessor never sees native messages directly.

This separation is intentional: the assessor needs a structured bird's-eye view for phase decisions, while the executor needs native tool context for implementation work.

8.7 Compaction in Native Mode

When scratchpad tokens exceed `compaction_threshold` after an execute phase:

1. `_compact_native_history()` is called with `compact_after` (protected zone boundary).
2. A split-safety guard walks backward from `compact_after` to ensure it does not bisect an assistant→tool_result group (would create orphaned tool_calls or dangling tool results).
3. Compactable messages are serialized to text and sent to the LLM via `complete_with_tools` (XML path) for summarization.
4. The compactable region is replaced by a single `<phase_summary>` user message.
5. `last_native_phase_start_idx` is set to `len(native_messages)` after compaction to avoid stale indices.

8.8 `complete_multi_turn` Method

Located in `mesh/llm.py`, this method:

- Accepts `messages: list[dict]` in OpenAI format.
- Passes them verbatim to the Chat Completions API (no XML serialization).
- Accepts pre-converted `tools: list[dict]` from `ToolRegistry.get_openai_tools()`.
- Handles reasoning effort negotiation (xhigh→high fallback, non-reasoning model stripping).
- Returns `(content, tool_calls, usage)`.
- `parallel_tool_calls` parameter (default `True`) controls whether the API is allowed to issue multiple tool calls per turn.

8.9 Protocol Events

The native loop emits the same protocol events as the XML loop:

- `emit_tool_call(name, arguments, call_id)` before each tool execution.
- `emit_tool_result(name, result, call_id, success)` after each tool execution.
- `emit_turn_started`, `emit_assistant_message`, `emit_thread_started/finished`, checkpoint events.

Chapter 9

Autonomous Project Controller

The v2 autonomous controller is distinct from the harness P&E controller. P&E chooses phases inside one execution task; autonomous agent mode advances a durable project across bounded sessions. It uses the ordinary router and worker-dispatch machinery, but adds trusted project scope, a project dossier, immutable session reports, persistent attempt budgets, and deterministic wake scheduling.

9.1 Enrollment and Trusted Session Scope

Enrollment is explicit in `mesh.yaml`. A node enables `autonomous_agent_mode_enabled`, lists the exact `autonomous_projects` it controls, and sets a per-session worker ceiling and active-mode pacing gap. The public example configuration shows the surface without shipping a live deployment.

The controller mandate is not placed unconditionally in every prompt. `AgentNode.autonomous_wake_metadata` recognizes the canonical `[AUTONOMOUS PROJECT SESSION]` header, validates the named `project:*` key against that node's enrollment, caps the requested session limit to configuration, and mints a session ID. Only this runtime-stamped metadata activates the planning mandate. A user message that merely imitates the header cannot enroll a project or mint a trusted session. Worker-report continuations receive the execute-and-close mandate rather than restarting the planning phase.

9.2 Durable Project Dossier

`mesh/project_dossier.py` defines the project state authority. Each dossier is a Markdown document under `~/.mesh/digests/` with frontmatter and exactly seven ordered, non-empty sections:

1. Identity,
2. Goals,
3. Tasks,
4. Timeline,
5. Narrative,
6. Standing decisions, and
7. Open threads.

Task rows live inside a schema-marked block and carry stable task IDs, pending/in-progress/done status, priority, goal linkage, and completion evidence. A validator checks identity, structure, the optional project-model block, and the global token ceiling before an exact-string edit lands. Writes use a same-directory temporary file, `fsync`, and rename; a failed validation leaves the prior dossier byte-identical. Initialization is create-once and never overwrites an existing dossier.

The controller is the sole routine dossier writer. Workers receive the dossier as read-only project context and return claims plus artifacts; they do not edit project state, write reports, or schedule more work. This separates execution evidence from the component that decides task transitions.

9.3 One Bounded Session

The shipped mandates (`mesh/prompts/autonomous_controller.txt` and `autonomous_controller_execute.txt`) define a two-phase protocol:

1. **Plan.** Check the daily ledger, read the dossier, follow only relevant report/memory links, repair the task roster, and write a bounded `SESSION PLAN`. The plan names a goal, ordered task IDs, completion evidence, and the first action before any worker launch.
2. **Execute and close.** Mark the selected task in progress, dispatch a self-contained worker brief, verify the resulting artifacts and tests, and continue only while both session and daily headroom remain. Direction-changing evidence is recorded in the dossier before another dispatch. Closeout writes the immutable report, indexes a compact memory pointer, updates live dossier state and the standing digest, relays the scheduler decision, and stops.

The router captures a schema-valid `SESSION PLAN` (four required fields, at most ten lines and 4,000 characters) on the outgoing planning turn. It stamps that bounded copy into trusted session metadata, then carries it on worker completion even if rolling conversation history has been summarized or dropped. A continuation without a valid carrier receives an explicit `SESSION PLAN UNAVAILABLE` recovery instruction rather than an invented plan.

9.4 Immutable Reports and Audit Chain

`dossier_write_report` creates a dated, sequenced report beneath `~/.mesh/reports/` and links it from the dossier Timeline. An identical retry is idempotent; different bytes at an existing report path are refused, so corrections require a later amendment report. The mandate requires the session plan, worker reports verbatim, controller verification, decisions, artifacts, issues, budget use, and next-wake intent. The report is the audit authority; the dossier is the current-state authority; episodic memory stores only a compact pointer back to the canonical report.

9.5 Admission Budgets and Single-Writer Boundaries

The daily project ledger is a JSON file beneath `~/.mesh/budget/`. Its limit comes from dossier frontmatter; the dated `used` counter rolls over on a new day. The central dispatch seam in `RouterV2` is the sole normal-path charger:

1. trusted session scope mechanically supplies a `[PROJECT: project:...]` tag when the model omitted it;
2. duplicate, capacity, and selection refusals run before the budget check and consume nothing;
3. an admitted dispatch is charged exactly once at reservation, before worker startup; and
4. a failed start is deliberately not refunded, because budgets bound attempts and must also bound retry storms.

The controller therefore reads `dossier_check_budget` but must never call `dossier_spend_budget`; doing so would double-charge the same attempt. Exhaustion returns a structured, final refusal.

A ledger-write failure after admission is logged as an understated count, and a missing tag or unreadable budget ledger fails open with a warning. Thus "one charge per attempt" is the validated normal path, while infrastructure failure is made observable rather than falsely represented as enforcement.

The operator can change the configured daily ceiling through the brokered control surface. The sibling `budget-reset` operation clears only the current day's `used` count; it does not change the limit. Dossier files, immutable reports, ledgers, and scheduled wakes are all stored on disk, so a process restart reloads rather than reconstructs project state.

9.6 Active Mode and Scheduled Wakes

Active mode is opt-in per project. After a successful report write, `AgentNode` calls the pure decision function in `mesh/autonomous_active.py`. It schedules no wake unless the dossier's active flag is true, daily budget remains, the last report is non-terminal, and no wake for that project is already pending. A minimum gap (60 minutes by default) paces sessions. If another wake for the same agent would overlap, the candidate is moved beyond it. The result is one explicit one-shot wake, not a recurrence; every closeout re-evaluates all gates from fresh state.

Reports with status `blocked`, `failed`, or `no_op` suppress automatic continuation. Pending wakes live in the agent's SQLite store and are reloaded at startup. The controller does not choose or override the runtime's timestamp: it reports the returned wake ID or the reason that no wake was scheduled.

9.7 Operator Surface

Autonomous controls use `AUTONOMOUS_CONTROL` messages routed through the central router and independently revalidated by the enrolled agent. The TUI exposes:

Command	Effect
`/auto`	Fan out status for enrolled controllers: projects, budgets, active flags, pending wakes, and current session.
`/auto wake <agent> [<project>] [<time>]`	Schedule a bounded project session; omitting time means immediate.
`/auto budget <project> <n>`	Set the daily worker ceiling (1–50).
`/auto budget <project> reset`	Clear today's spent-attempt count while retaining the ceiling.
`/auto active <project> on|off`	Arm or disarm paced automatic continuation.
`/auto cancel <agent> <wake-id>`	Cancel a pending wake.
`/wake [project]`	Convenience form that resolves the project's owner and starts a session immediately.
`/report [project] [since YYYY-MM-DD]`	Request a report-oriented autonomous writing session.

Table 9.1: Autonomous controller operations in `run_user_tui.py`.

Chapter 10

Context Management

10.1 Forced Synthesis

Forced synthesis is a budget-exhaustion mechanism in the *inner* TAOR loop (`run_loop()`).

$$\texttt{synthesis\_threshold} = \lfloor \texttt{soft\_limit} \times 0.97 \rfloor$$

At each iteration, `estimate_history_tokens(history)` is compared against the threshold. When exceeded:

1. All tools are stripped (`effective_tool_names = []`).
2. The instructions are replaced with a forced-synthesis message directing the model to produce its final response immediately.
3. A budget line is prepended showing current and maximum token counts.
4. An error event is emitted (non-fatal).

The budget line is injected on *every* iteration, even before forced synthesis triggers, giving the model continuous awareness of context consumption.

10.2 Compaction

10.2.1 When It Fires

Compaction runs *only after execute phases*. It fires when:

$$\texttt{scratchpad\_tokens} = \texttt{estimated\_tokens} - \texttt{initial\_tokens} > \texttt{compaction\_threshold}$$

where:

$$\texttt{workspace} = \max(\texttt{soft\_limit} - \texttt{initial\_tokens},\ 50{,}000)$$
$$\texttt{compaction\_threshold} = \lfloor \texttt{workspace} \times 0.40 \rfloor$$

No compaction after observe phases. This was an intentional fix (commit `d8d74ca5`) to preserve raw observation data for the planner.

10.2.2 Protected Zone

`compact_after` marks the boundary between protected and compactable history. It is set to the initial history length and never moves. Everything before `compact_after` (the original task prompt) is never touched.

10.2.3 Compaction Procedure

The function `_compact_history()`:

1. Concatenates all messages after `compact_after` into a single string.
2. Formats `_COMPACTION_TEMPLATE` with the phase prompt and appends the raw text.
3. Calls the primary LLM (not the assessor) with a fresh single-turn conversation.
4. Extracts content from `<phase_summary>` tags if present.
5. Deletes all messages after `compact_after` and inserts a single summary message.

The compaction template instructs the LLM to preserve five categories: actions attempted, successes and key facts, failures with verbatim error messages, artifacts produced, and open sub-questions.

Failure mode. If the compaction LLM call fails, the first 2000 characters of `executor_output` are preserved as a fallback.

10.3 Degenerate Loop Detection

The checkpoint system detects and responds to degenerate executor behavior (reading the same files repeatedly without progress). It operates as a *side-channel*—checkpoint Q&A is NOT appended to the main conversation history.

10.3.1 Tracker State

Maintained per `run_loop()` invocation:

Variable	Purpose
`_ckpt_has_written`	Set `True` on first write tool call
`_ckpt_read_only_streak`	Consecutive turns with only read tools
`_ckpt_last_tool_sig`	Sorted tuple of (name, args) from last turn
`_read_tools_stripped`	Whether read tools are currently removed
`_consecutive_degenerate`	Count of consecutive degenerate checkpoints

10.3.2 Checkpoint Triggers

Three conditions fire a checkpoint, in priority order:

1. **post_write_stall:** A write has occurred (`_ckpt_has_written`) and `_ckpt_read_only_streak >= 3`.
2. **duplicate_read:** The current tool signature matches the previous turn's signature exactly.
3. **periodic:** `iteration >= start` and `(iteration - start) % interval == 0`. Default: `start=8`, `interval=8` (configurable per caller; P&E uses `interval=5`, `start=8`).

10.3.3 Three-Level Escalation

When `degenerate = True`:

Reset conditions.

- `completed = True` resets `_consecutive_degenerate` to 0.
- A write tool call resets `_consecutive_degenerate` to 0 and restores read tools if stripped.
- Neither completed nor degenerate (both NO) resets `_consecutive_degenerate` to 0.

Level	_consecutive_degenerate	Action
1	1	Strip read-only tools (file_read, list_dir, grep, find_files) from subsequent iterations. Restored on next write.
2	2	Strip all <reasoning> tags from history via regex. Reduces context size by removing chain-of-thought tokens.
3	≥ 3	Force-terminate the phase with an error message.

10.4 CC Watchdog

The CCWatchdog class monitors Claude Code (claude-code) and zai backend executor sessions.

10.4.1 Architecture

Unlike checkpoints (which are fired by the inner TAOR loop), the watchdog is a callback invoked by the CC/zai streaming integration every 8 CC turns. It receives new turn batches and accumulates the full phase history internally.

10.4.2 Evaluation

Each watchdog call:

1. Builds evaluator context XML with a <recent_activity> demarcation for the last RECENT_WINDOW=16 turns.
2. Truncates oldest turn results if context exceeds WATCHDOG_MAX_CONTEXT_TOKENS = 100,000.
3. Calls the assessor LLM with _WATCHDOG_PROMPT asking YES/NO on progress.

10.4.3 Kill Logic

- **YES** (progress): reset _consecutive_no to 0.
- **NO** (stuck): increment _consecutive_no.
- **Kill** when _consecutive_no >= 2.

On assessor call failure, the watchdog defaults to "continue."

10.5 In-Flight Context Pruning

_manage_in_flight_context() prunes old tool results in the standard TAOR loop when estimated tokens exceed soft_limit$\times$0.8. It keeps the most recent KEEP_RECENT_RESULTS=4 in-flight messages and inserts a [N previous tool result(s) omitted] marker.

10.5.1 Extreme Result Truncation

_truncate_extreme_result() caps any single tool result at soft_limit $\times$ 3 characters.

10.5.2 Tool Result Size Gating

In `_execute_tool_calls()`, individual tool results exceeding `soft_limit` $\times 0.4$ characters trigger a size gate (the result is truncated with a marker).

10.6 Codex Assessor

As an alternative to the LLM-based assessor (which uses `complete_with_tools` via the standard API path), the harness supports a **codex CLI subprocess** as the assessor controller. This is implemented in `_run_codex_assessor()` in `mesh/harness/loop.py`.

Motivation. The codex CLI provides built-in reasoning (via OpenAI's `o3` or similar models) with a simpler interface—no tool schema management or XML parsing required. It was introduced as a second attempt after a clean revert (commit `54a33f8a`) of the original implementation.

Invocation. The function builds a subprocess command:

```
codex exec --model <model> -s read-only --ephemeral --json \
    --config "reasoning_effort=<effort>" -C <cwd> -
```

Flags: `--ephemeral` (no persistent state), `--json` (JSONL output), `-s read-only` (sandbox), and `-` (read prompt from stdin). The prompt is the concatenation of the evaluator context XML and the controller instructions template—the same `_CONTROLLER_TEMPLATE` used by the LLM assessor.

Execution. The subprocess is launched via `asyncio.get_running_loop().run_in_executor(None, ...)` to avoid blocking the event loop, wrapped in `asyncio.wait_for` with a `CODEX_ASSESSOR_TIMEOUT` of 300 seconds (plus 10s grace for the outer wait).

Output parsing. The codex CLI with `--json` emits JSONL events. The parser looks for two event patterns:

1. Events with `type=message, role=assistant`: extracts the `output_text` content part.
2. Events with `type=response.completed`: extracts assistant message content from the nested `response.output` array.

If neither yields structured content, the parser falls back to raw stdout—checking for `<intent>` tags directly in the output.

Failure handling. On timeout or non-zero exit code, the function returns a synthetic `<intent>terminate</inte` response with an error summary. This ensures the P&E controller always receives a parseable intent, even on infrastructure failure.

Configuration. Three fields on `LLMBackendConfig` control the codex assessor: `harness_codex_assessor` (bool, enables it), `harness_codex_assessor_model` (default: `"o3"`), `harness_codex_assessor_effort` (reasoning effort string), and `harness_codex_assessor_binary` (path to the codex CLI binary, default: `"~/.local/bin/codex"`).

10.7 Known Discrepancies and Issues

1. **Phase discipline is advisory.** The observe phase restricts tools via `tool_names` but CC/zai backends have their own tool surfaces. The harness cannot prevent a CC executor from editing files during an "observe" phase.

2. **Assessor token overhead.** Operational data shows the assessor consuming up to 65.7% of total token budget. Each phase transition requires a full evaluator context XML document.

3. **Assessor hallucination.** The assessor (a toolless LLM call) frequently attempts to emit tool calls. The harness retries up to `HALLUCINATION_MAX_RETRIES=2` times.

4. **Triage is dead code in P&E path.** `_TRIAGE_TEMPLATE`, `parse_triage()`, and `TriageDecision` exist but are not called by `_run_plan_and_execute_loop()`.

5. **Legacy plan infrastructure.** `PLAN_INSTRUCTIONS`, `extract_plan_and_phase_prompt()`, and `_ASSESSMENT_TEMPLATE` are remnants of the fixed-sequence controller and are not invoked.

6. **Compaction uses primary LLM, not assessor.** `_compact_history()` calls the executor's LLM, not the assessor—potentially using an expensive model for summarization.

7. **400 treated as transient.** `TRANSIENT_HTTP_CODES` includes 400 (Bad Request), which is typically a permanent client error, not transient.

Part III

Memory and Knowledge

Chapter 11

Memory System

The v2 memory system gives agents persistent recall at several levels of abstraction. Its main text hierarchy is raw conversation history, three-tier episodic records, an always-on standing digest, and cited entity essays. An entity registry binds records to people, projects, events, and optionally groups. A bounded edit-based curation turn maintains the digest and essays after formation. Governed procedural memory is a separate layer: human-approved skill cards describe how to act, rather than what happened. Project maps remain an optional legacy subsystem and are not part of the self-curation hierarchy.

11.1 Architecture Overview

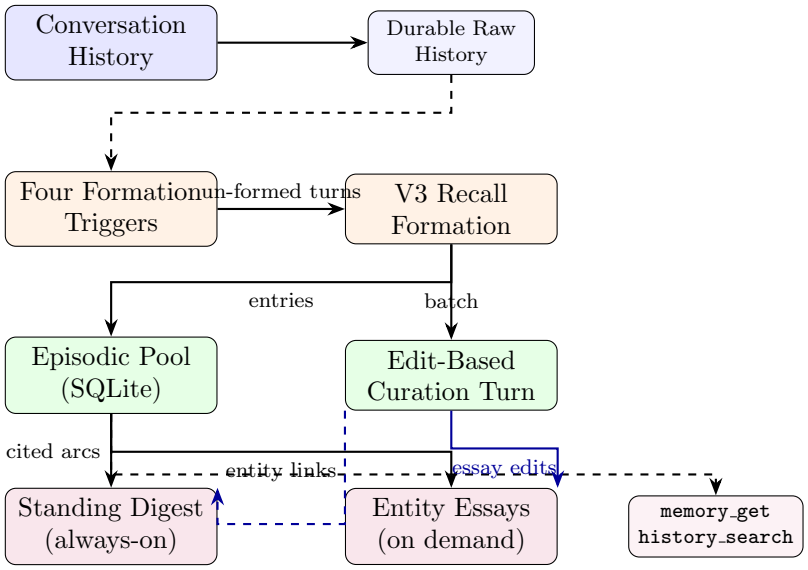

Figure 11.1: Current memory data flow. Startup, periodic, token-pressure, and shutdown triggers feed V3 formation. Records commit to the episodic pool; each successful batch queues a scoped curation turn that edits the standing digest and active-entity essays. The raw log and episodic store remain searchable beneath the curated text hierarchy.

Text hierarchy and governed procedure layer.

Conversation history The durable raw message record and rolling context window. Context compaction may summarize the active window, but Formation V3 consumes its own cursor over un-formed turns rather than treating window drops as a formation trigger.

Memory pool Episodic memories stored in a SQLite database (`~/.mesh/memory/{agent}.db`). Each entry has three tiers of detail (Table 11.1). The pool persists across sessions and grows over the agent's lifetime.

Standing digest One per-agent Markdown briefing beneath `~/.mesh/digests/`. It is injected in place of the entry-level TOC and carries cross-project chronology, interpretation, durable decisions, live threads, and an agent narrative.

Entity essays Cited narratives for active person, project, and event entities, stored in the per-agent SQLite database. They merge many linked episodes into a coherent, entity-specific account and are fetched through `essay_list`/`essay_get`.

Skill cards Versioned YAML procedures under `~/.mesh/skills/`. Cards are selected separately from historical recall and are advisory until a human approves their lifecycle state.

Tier	Name	Content
1	Summary	Short retrieval key (1–2 sentences). Always visible in the TOC.
2	Reflection	Detailed analysis: what happened, why it matters, lessons learned. Fetched via `memory_get(id)`.
3	Trace	Full tool call trace from the original conversation. Fetched via `memory_get(id, full=true)`.

Table 11.1: Three-tier memory entry structure. Higher tiers are fetched on demand to avoid context bloat.

11.2 Formation V3 Pipeline

Memory formation converts un-formed conversation turns into structured episodic entries. The V3 pipeline uses a dedicated LLM (`memory_llm_backend`) that is decoupled from the agent's primary worker backend. `mesh/memory/formation_contract.py` is the single prompt and parse authority shared by live formation and fold-compatibility tooling.

Trigger and timing. Formation fires on four independent triggers (the defaults below are configuration values, not provider requirements):

1. **Startup**: after memory initialization and embedding migration.
2. **Timer**: every 1,800 seconds, deferring a tail younger than 300 seconds so an exchange can finish.
3. **Token pressure**: after roughly 30,000 newly appended, uncommitted tokens.
4. **Shutdown**: one final blocking attempt with a 30-second cap.

Under V3, `MemorySystemV2.on_window_drop()` explicitly does *not* form memories; it only updates rolling conversation-summary state. This decoupling prevents context-compaction policy from controlling durable memory formation.

Segmenter. The V3 segmenter (`mesh/memory/formation_v3.py`) sends bounded, overlapping windows of un-formed turns to the configured memory LLM. Defaults are 60 turns per window with 20 turns of overlap and a 10-turn in-window defer tail. The LLM returns a JSON array of records under the shared recall-oriented contract. Each record contains:

- `topic_label` — concise label for the segment.
- `summary`, `reflection`, and `trace` — the three episodic detail tiers.
- `retrieval_key` — compact search surface.
- `project` — project assignment. The prompt encourages grounded labels but does not depend on a project map.
- `tags` — categorization tags.
- `outcome` — success, partial, or failure.
- `event_date` — the event's best date, distinct from the formation timestamp.
- `digest_candidate` — the second-bar signal described below.
- optional entity-resolution metadata when shadow or write mode is enabled.

Two bars. Formation is deliberately recall-oriented: it does not require or filter on a 1–10 "worthiness score." Broadly useful events enter the episodic pool. The higher precision bar is the boolean `digest_candidate`; curation uses it to decide which records may change always-on narrative context. This separates "worth remembering" from "worth occupying every future prompt."

Cursor mechanics. A cursor index tracks which conversation turns have been processed. Entry persistence and cursor advancement are atomic—if persistence fails, the cursor stays put and the window is retried on the next trigger. A cursor clamp prevents the cursor from advancing past the actual history length after restarts. Empty-but-valid extraction may advance the cursor, because it is a completed judgment rather than a parse failure.

Failure tolerance. A configurable strike counter tracks parse failures for one window. At the default threshold of three, the system writes a visible placeholder entry tagged `formation-fallback`, then advances. The failure therefore does not wedge formation and is not silently erased as an ordinary "skip."

Cold-start handling. When project maps are disabled, formation continues normally: project assignment is a field on each episodic record, not a dependency on map creation. If the optional map subsystem is enabled, the retained compatibility path may bootstrap and integrate maps after entry persistence.

11.3 Edit-Based Self-Curation

After a successful formation commit, `RouterV2` enqueues one internal curation batch. A serialized drain loop processes those batches outside the normal message lock, so an LLM maintenance call cannot starve interactive traffic. The curation turn receives the new batch, the full active/pending entity registry, bounded artifact measurements, and any durable additions that a previous turn could not land.

Scoped update loop. The internal turn uses the existing router tool loop with a deliberately narrow catalog. It may inspect memories, entities, essays, and the digest; create/link/merge/retire entities; correct entity links; and apply exact `digest_edit` or `essay_edit` replacements. Group

mutations are present only when the group feature is enabled. Every mutation runs under a curation execution context (shadow or write mode), so normal interactive text cannot silently acquire internal curation authority.

The turn has three responsibilities:

1. link the new records to existing entities or create grounded pending entity proposals;
2. maintain essays for active entities using all linked evidence, not only the latest batch; and
3. update the standing digest only with high-bar, agent-wide material, preserving exact memory citations.

Digest constitution. `DIGEST_CONSTITUTION` in `mesh/memory/curation.py` is the live write authority. The digest must retain seven non-empty sections: Timeline, Narrative, Projects, People, Standing decisions & conventions, Open threads, and Agent narrative. Timeline is chronological; Narrative is causal and compressed by liveness; Standing decisions grow monotonically; Open threads are rewritten as live state. Every `[m_<12hex>]` citation must resolve and support the particular claim it follows. The tracked companion source `mesh/fold_driver/prompts/standing_digest/constitutio` supplies the shared standing-digest rules used by formation/fold compatibility.

Essay constitution. Entity essays are the substantive narrative layer, not registry metadata. Each entity type has a required structure; claims cite only records linked to that entity. Publication is restricted to active entities and validates citations, placeholders, evidence scope, concurrency version, and token budget. Initial essay generation uses the same publish path and gets one bounded repair attempt; ongoing changes use `essay_edit`.

Nightly fold status. The old offline nightly fold is no longer the primary maintenance writer. Digest and essay changes now land during these edit-based curation turns. `mesh/fold_driver/` and `mesh/memory/essay_fold.py` remain as tracked compatibility, bootstrap, and audit tooling, but a deployment should not describe the retired chronological batch fold as the live digest path.

11.4 Entity Registry and Cited Essays

`mesh/memory/entities.py` owns a per-agent SQLite registry. Entity keys are typed as person, project, event, or group and move through pending/active/retired states. Aliases are Unicode-normalized; merges retain a replacement key; memory links and correction events are transactional. Activation is evidence-based: by default an entity must be observed across three distinct formation windows, rather than repeated several times in one window. The active-entity cap bounds registry growth.

The entity service is also the publication gate for essays. Before a write commits it checks that the target is active, the evidence version is current, every canonical memory ID resolves, every cited memory is linked to the target, no loose or placeholder citation syntax remains, and the body fits its ceiling. `mesh/memory/ids.py` normalizes accepted memory-ID surfaces to the canonical 12-hex handle. These gates apply equally to generated and hand-edited essays; no alternate write path bypasses them.

11.5 Ceilings, Durable Carry-Forward, and Write Audit

Curated artifacts have hard token ceilings. `ceiling_rules.py` implements a make-room-then-land contract: an over-ceiling write is refused, never truncated, and the refusal carries an artifact-

specific compaction order plus an instruction to retry the same addition in the same turn. Digest compaction merges old quiet Timeline ranges first, compresses Narrative by liveness second, and tightens must-keep sections last without emptying them or discarding the only surviving judgment. Essay compaction similarly preserves cited conclusions over event-by-event detail.

If the model cannot land the addition in that turn, `pending_additions.py` writes the full, unclipped edit into the agent's SQLite database at turn end. Up to eight oldest pending additions are offered back on a later curation turn. A row resolves only when exact committed text proves it landed, or when citation-preserving markers prove a compressed replacement superseded it. The durable queue is bounded per artifact, rejects lossy previews, and survives restarts with the rest of the memory store.

`write_audit.py` records every curated-artifact write attempt in the existing entity-event trail. Outcomes distinguish landed, refused, retried, and queued attempts and include before/after hashes, token measurements, and retry attribution. The audit can therefore compute retry success, terminal drops, and carried-forward additions without changing the write decision.

11.6 Retrieval

Several retrieval paths expose different abstraction levels instead of forcing every query through one index.

Always-on digest or TOC. When `standing_digest_enabled` is true, the validated digest is injected in place of `<memory_toc>`. If no standing digest is enabled, the fallback TOC contains up to 30 representative episodic records. The two-stage TOC algorithm remains implemented and is described in Chapter 12.

On-Demand Tools.

- `memory_get(id)` — fetches the full reflection (Tier 2) of a specific entry. The result persists in conversation history for the rest of the session.
- `memory_search(query, project=...)` — defaults to hybrid search: embedding similarity and SQLite FTS5/BM25 lexical rankings are merged by reciprocal-rank fusion. Explicit embedding-only and lexical-only modes remain available.
- `history_search(query)` — keyword search over durable raw conversation logs for details that were never formed or were omitted from a higher-level summary.
- `digest_get`, `essay_list`, and `essay_get` — retrieve curated context by agent or entity before opening granular episodes.

Router-Level Injection. Before dispatching work to a worker, the router runs `_select_memory_context()` to curate and inject relevant memories into the worker's context. A small relevance-filtered set is formatted as `<router_injected_context>` XML and prepended to the worker prompt. Optional project-map context may be appended only when that legacy subsystem is enabled.

Correction tools. The memory layer is mutable under explicit, audited operations. An ID-stable `memory_edit` re-embeds changed retrieval fields; `memory_delete` removes an episode and updates selection state; `digest_get`/`digest_edit` maintain the standing briefing; and essay edits run through entity citation and ceiling validators. Corrections therefore change the agent-facing artifacts rather than merely appending a contradictory raw-history message.

11.7 Feature Flag Matrix

Memory features are deployed incrementally per agent via configuration flags in `mesh.yaml`. Formation V3 uses a dedicated LLM backend (`memory_llm_backend`) decoupled from the agent's worker backend. The current code independently gates V3 formation, standing-digest injection, entity resolution (off/shadow/write), self-curation mode, essay generation, groups, and legacy project maps. Configuration validation refuses self- curation enrollment unless V3 formation and the standing digest are enabled and project maps are disabled; this prevents two narrative writers from competing for the same role.

11.8 Standing Digest (Curation-Maintained)

The standing digest extends the memory system with a dense, continuously updated summary document per agent. Where the episodic memory pool stores individual entries with three-tier retrieval, the standing digest is a single coherent narrative—a "standing briefing"—that captures the agent's full operational history, key decisions, project context, and relationship knowledge in one document.

Architecture. The live writer is the curation turn in `mesh/memory/curation.py`. It proposes exact edits after successful formation batches; `AgentNode` validates section structure, citations, the measured token ceiling, and write authority before an atomic update. Over-ceiling refusals must compact and retry or enter the durable pending-additions ledger. The old cursor-driven offline fold remains available for compatibility and bootstrap work, not as the normal nightly maintenance path.

Configuration. The live surface is centered on:

- `standing_digest_enabled` (bool) — when true, the published digest replaces the `<memory_toc>` block in the agent's system prompt.
- `standing_digest_path` (str) — path to the published digest file (Markdown).
- `entity_self_curation_mode` — off, shadow, or write mode for internal mutations.
- curation tool/iteration, essay, group, and failure-alert limits — bounded independently from normal message turns.

Quality gates. Each write checks citation provenance (every `[m_...]` handle resolves), the seven required non-empty sections, exact replacement semantics, and the measured ceiling. Curation instructions add claim-to-citation correspondence, chronological Timeline ordering, recent-coverage review, and graduation rules that forbid deletion of the only surviving judgment. Write auditing records both successful and refused attempts.

Relationship to episodic memory. The standing digest and the episodic memory pool are complementary, not competing. The pool stores granular entries for on-demand retrieval; the digest provides always-on holistic context. When `standing_digest_enabled` is true, the digest replaces the TOC injection (which would otherwise list individual entry summaries), while the pool's search and retrieval tools remain available.

11.9 Governed Procedural Memory

`mesh/procedural_memory.py` stores procedures as versioned YAML *skill cards*, separate from episodic facts and entity narratives. A card records triggers, typed preconditions, authority, source files and SHA-256 fingerprints, required invariants, verification, rollback, evidence, and an append-only outcome history.

The lifecycle is `proposed` → `active` → `retired`, but the runtime deliberately exposes no activation API. `skill_draft` can package a completed worker trace and source evidence into a proposal; a human must review the file and supply approval metadata before it becomes active. Source-fingerprint drift suppresses stale cards.

Selection is deterministic and advisory. Required typed preconditions are checked for contradictions first; contradicted cards are excluded. Eligible cards are then ranked by BM25-like text relevance with stricter treatment of unknown required facts, and at most three are injected. The rendered block explicitly grants no additional authority and never auto-executes a procedure. Outcome receipts support later meta-review without converting observed success into automatic policy.

11.10 Key Files

File	Purpose
mesh/memory/system_v2.py	Core formation orchestration, episodic retrieval, TOC/FL selection, correction operations, and optional map compatibility.
mesh/memory/formation_v3.py	V3 segmenter: JSON-structured LLM segmentation with validation, clamping, and retry logic.
mesh/memory/formation_contract.py	Shared low-bar prompt, canonical record fields, entity envelope, and response parser.
mesh/memory/store.py	SQLite persistence: `MemoryEntry` dataclass, CRUD operations, FTS5 index, entities, essays, pending additions, wakes, and maps.
mesh/memory/entities.py	Per-agent entity registry, link correction, merge/retirement, preflight validation, and cited-essay publication gates.
mesh/memory/entity_essays.py, essay_fold.py	Canonical essay generation/publication plus retained fold compatibility.
mesh/memory/curation.py	Scoped edit-based curation batches, digest/essay constitutions, shadow and write execution contexts.
mesh/memory/ceiling_rules.py	Artifact-specific make-room-then-retry refusals.
mesh/memory/pending_additions.py	Durable queue and evidence-based resolution for un-landed additions.
mesh/memory/write_audit.py	Per-attempt outcome, hash, token, retry, and drop-rate audit records.
mesh/memory/ids.py	Canonical 12-hex memory-handle normalization.
mesh/memory/selection.py	FLMI active set selection using embedding vectors.
mesh/memory/embeddings.py	Embedding client for memory entry vectorization.
mesh/procedural_memory.py	Governed YAML skill-card schema, human lifecycle, selection, receipts, source-drift suppression, and proposal drafting.
mesh/paths.py	Central path resolution for memory, digests, maps, skills, and isolated agent state.
mesh/prompts/memory.md	Shared prompt instructions for memory, digest, essay, and history tools.

Table 11.2: Memory system source files.

Chapter 12

TOC Selection and Optional Project Maps

12.1 Compatibility Status of Project Maps

Project maps remain implemented, but they are an optional legacy narrative subsystem rather than a tier in the v2 self-curation hierarchy. When enabled, maps are Markdown files beneath `~/.mesh/memory/maps/` with summary/embedding metadata in SQLite. Formation can asynchronously bootstrap an unknown project and integrate a multi-project entry batch through full-section replacement or section append operations. Integration failure is non-blocking with respect to episodic persistence and cursor advancement.

The retained canonical map schema is Summary, Goals, Architecture, Key Files, Key Decisions, Current State, Open Issues, and Glossary. Self-curation enrollment validation requires `project_maps_enabled=false`, because the curation-maintained digest and project/entity essays now own the durable narrative role. Deployments that keep maps enabled may still use the routing and selection mechanics below.

12.2 Relevance-Based Map Injection

When `project_maps_injection_enabled` is true, project maps are injected into router and worker prompts based on relevance to the current conversation, replacing the earlier approach of injecting only the statically tracked `_active_project` map.

Algorithm.

1. Concatenate the text of the last N conversation turns (N=5 by default, hardcoded in the router's `_get_last_n_turns_text()` helper).
2. Embed the concatenated text using the memory embedding client (`text-embedding-3-small`).
3. Compute cosine similarity between the context embedding and each project map's summary embedding (stored in the `project_maps` table).
4. Return the top-k maps (k=2) with similarity above a minimum threshold (0.2).

Injection points.

- **Router prompt**: `_build_system_prompt()` and `_build_router_prompt()` call `render_relevant_maps_bloc` (which wraps `select_relevant_maps()`), injecting up to 2 maps as `<project_map project="..." relevance="...">` blocks.

- **Worker prompt**: `_select_memory_context()` appends relevant maps to the `<router_injected_context>` block alongside memory entries. Workers previously received no project maps.
- **Fallback**: if no map embeddings are available (embedder down, or maps lack summaries), falls back to `render_maps_block()` (active project only).

Every path first checks the map-injection feature gate; disabled means an empty block, not an active-project fallback.

Summary embeddings. Each project map maintains a 1–2 sentence summary in the `project_maps.summary` column, extracted deterministically from the map's `## Summary` section after every content change. The summary is embedded via `text-embedding-3-small` and stored in the `embedding` BLOB column. Summaries are better embedding targets than full map content: they capture the project's domain and focus in a dense, semantically rich form, whereas full maps contain structural/list content that dilutes embedding quality.

Design rationale. The static `_active_project` approach had two limitations: (1) only one map was ever injected, even when conversations spanned multiple projects, and (2) workers received no project context at all. Relevance-based selection aligns map injection with the same principle used for memory entry retrieval.

12.3 TOC Selection and Weighting (FLMI)

When the standing digest is disabled, up to 30 entries shown in the prompt TOC are selected from the full pool using a two-stage pipeline that balances breadth with diversity. With a standing digest, this remains a fallback and diagnostic mechanism rather than the normal always-on prompt block.

Stage 1: Cosine relevance. The top 100 entries by cosine similarity to the last N conversation turns are selected as candidates. This ensures the TOC is grounded in the current conversational context.

Stage 2: Facility Location diversity. From the top-100 candidates, the `lazy_greedy_fl()` function (in `mesh/memory/selection.py`) selects 30 entries using a submodular Facility Location objective. This maximizes embedding diversity—ensuring broad coverage of the agent's topic history rather than clustering around a single recent topic.

Representative-set distinction. The older persistent active-set selector can starve a redundant new entry of facility-location gain. The query-conditioned TOC path does not treat that active set as a pool ceiling: it first restricts the full candidate pool by the current query, then runs `lazy_greedy_fl()` inside that top-100 set. This preserves the useful diversity objective without describing the legacy active set as the whole memory hierarchy.

Summary maintenance. When the optional subsystem is enabled, each project map has a short summary (1–2 sentences) stored alongside an embedding vector in the `project_maps` table. Summaries are extracted deterministically after every map content change—via integration, tool edits, or manual curation—by parsing the canonical `## Summary` section from the map content (no LLM call needed). If no `## Summary` section exists, the first non-heading paragraph (up to

500 characters) is used as a fallback. The embedding is recomputed from the updated summary and cached in memory. The update runs as a fire-and-forget `asyncio.create_task`.

Part IV

Infrastructure and Operations

Chapter 13

Multi-Host Deployment

The mesh system supports multi-host deployments where a central router coordinates agents running on separate machines. This chapter documents the deployment topology patterns, service management, and cross-host coordination mechanisms. See also `docs/two-host-demo.md` for a concrete walkthrough.

Typical topology. A production deployment assigns distinct roles to hosts:

Primary host Runs the router (optionally systemd-managed), multiple agents, an nginx reverse proxy with TLS termination, and the web client.

Secondary host(s) Run additional agents. Connected to the primary via WebSocket (optionally through an nginx proxy for TLS).

Backup target (optional) Receives daily incremental rsync snapshots from the primary host.

Service management. The router and critical agents can be managed via systemd units with automatic restart on failure. Other agents are managed via `agent-ctl.sh`, a shell script providing start/stop/status operations with backend selection.

Cross-host sync. Syncthing or similar tools can synchronize `~/.mesh` and shared directories across hosts. This ensures memory databases and configuration changes propagate automatically. The application code can be synchronized via git.

Backup. A daily incremental rsync job can copy the mesh state to a backup target with configurable retention.

Chapter 14

Security Hardening

Several hardening measures protect subprocess spawning, credential handling, and inter-process communication.

14.1 Subprocess Environment Allowlist

All LLM subprocess methods (`_run_cc_subprocess`, `_complete_codex`, `_complete_mesh_harness`, `cc_session.py`, `router_cc.py`, `eval/evaluator.py`) filter `os.environ` through `_build_subprocess_env()`, a module-level helper in `mesh/llm.py`. The allowlist is a `frozenset` of 30 approved environment variables plus prefix matching for `CLAUDE_*` and `ANTHROPIC_*`.

Category	Allowed Variables
System	`PATH`, `HOME`, `USER`, `SHELL`, `LANG`, `LC_*`, `TERM`, `TMPDIR`, `XDG_*`, `SSH_AUTH_SOCK`
API keys	`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `DEEPSEEK_API_KEY`, `EXA_API_KEY`, `ZAI_API_KEY`, `OPENROUTER_API_KEY`, `SYNTHETIC_API_KEY`, `GOOGLE_API_KEY`
Mesh	`MESH_AUTH_TOKEN`, `MESH_NODE_ID`, `MESH_ATTACHMENT_SECRET`
Python	`VIRTUAL_ENV`, `PYTHONDONTWRITEBYTECODE`, `PYTHONHASHSEED`

Table 14.1: Subprocess environment allowlist categories. Variables not in this list (e.g., `LD_PRELOAD`, `LD_LIBRARY_PATH`, `PYTHONPATH`) are blocked.

Operator-controlled overrides (`cc_env`, `env_override` in `LLMConfig`) are applied *after* filtering and may inject arbitrary variables by design (e.g., account-specific `HOME`, custom API keys).

14.2 API Key Masking in Logs

The `_mask_cmd()` helper in `mesh/llm.py` masks sensitive values in subprocess command logs. It handles both two-token (`--api-key VALUE`) and single-token (`--api-key=VALUE`) forms, replacing values with `***` plus the last 4 characters. Only log output is affected; the actual subprocess command is unchanged.

14.3 Socket Path Hardening

Agent tool sockets moved from the world-readable `/tmp/` directory to `~/.mesh/sockets/` with directory mode 0700 (owner-only). Socket names are deterministic (`{sanitized_node_id}.sock`) within the restricted directory— no random suffix needed since only the owning user can list or connect.

14.4 Node ID Propagation

`MESH_NODE_ID` is set via two independent paths:

1. **Process-level**: `os.environ["MESH_NODE_ID"]` set in `AgentNode.__init__` for tool subprocess inheritance.
2. **Per-LLMClient**: `LLMConfig.node_id` injected explicitly into each LLM subprocess environment. Ensures correctness if multiple `LLMClient` instances exist in one process.

14.5 Agent Node Refactoring

The 842-line `_process_with_llm` method was decomposed into a 280-line orchestrator plus six focused helpers:

- `_setup_controller_for_message` — controller initialization and streaming observer.
- `_build_system_prompt_for_llm` — memory, preferences, personality, trace prompt assembly.
- `_build_worker_instructions` — worker instructions and briefing mode setup.
- `_handle_controller_llm_response` — controller decision routing.
- `_dispatch_special_tool_calls` — 10 special tool types (send_message, schedule_wake, etc.).
- `_process_tool_calls_in_loop` — full tool iteration processing with retry and error handling.

14.6 Confirmation Gate Bypass

Two mechanisms allow tool calls to skip interactive confirmation:

1. **Socket handler** `skip_confirmation=True` (`agent_node.py`): When a worker subprocess calls a confirmation-required tool (e.g., `gmail_send_message`) via the agent socket, the socket handler executes it with `skip_confirmation=True`. The rationale is that the user already authorized the work by dispatching the worker—re-prompting for confirmation on each sub-action would make the worker unusable.
2. `auto_confirm_tools` **config**: Per-agent YAML configuration that lists tool names which should always skip confirmation for that agent. Used for Alice's Gmail write tools in production.

Both mechanisms are intentional design decisions, not security bypasses. The trust boundary is at worker dispatch time, not at individual tool invocation time within an authorized worker session.

14.7 IDLE→BUSY Race Fix

In `router_v2.py`, the worker task is now created *before* `self._state` is set to `BUSY`. Since asyncio is single-threaded, the task does not execute until the next yield, but the invariant `_state == BUSY` $\implies$ `_worker_task is not None` now holds at every point.

Chapter 15

Benchmark Infrastructure

The mesh includes a benchmark harness for systematic evaluation of backend configurations against a curated test suite. This chapter documents the infrastructure; results are presented in Chapter 16.

Runner. `benchmark/run_sanity_mesh_harness.py` is the main entry point. It accepts a `--mesh-backend` argument specifying which `mesh.yaml` backend to evaluate, plus optional flags for thinking budget, assessor effort, and codex assessor configuration. The runner iterates over a suite of test instances, launching the harness subprocess for each, collecting patches, and running evaluation (typically `pytest` inside a Docker container matching the target repository).

Backend resolution. `benchmark/_mesh_backend.py` defines `ResolvedBackend`, a dataclass that resolves a backend name from `mesh.yaml` into a complete configuration including model, API endpoint, controller mode, soft limit, thinking budget, and assessor configuration.

Test suites. The primary suite is `benchmark/delta_suite.json`, a 15-instance collection covering 10 hello-world instances (drawn from real mesh system bugs) and 5 SWE-bench instances (astropy, django, ansible, etc.). Additional suites exist for research benchmarks and full SWE-bench evaluation.

Evaluation. Each instance includes a test specification. After the harness produces a patch, the runner applies it to a clean checkout inside a Docker container and runs the test suite. Results are recorded as pass/fail counts per instance, along with token usage, wall time, and iteration counts.

Part V

Evaluation

Chapter 16

Benchmark Results and Ablations

This chapter will present empirical results from benchmark evaluations of different backend configurations, including ablations over thinking budgets, assessor effort levels, and controller modes. Key comparisons include:

- **Standard vs. P&E mode** on the 15-instance delta suite, with per-instance breakdowns showing where the assessor adds value and where it regresses.
- **Thinking budget ablation** (Qwen3.6-27B with and without `thinking_budget: 16384`): standard mode improved from 9/15 (60%) to 11/15 (73%) with thinking enabled, while P&E held at 12/15 (80%) with different failure composition.
- **Heterogeneous backend combinations**: local Qwen executor with DS-pro API assessor, GLM-5.1 executor with DS-pro assessor, and monolithic API-only configurations.
- **Wall time and cost analysis**: standard + thinking at 82 min vs. P&E at 220 min for similar accuracy on a free local model.

Chapter 17

Lessons Learned

This chapter documents operational lessons from running the mesh system in production across multiple agents for 5+ months.

- **Memory retrieval adoption**: Despite a well-populated memory pool (450+ entries across agents), active retrieval (`memory_get`, `memory_search`) remained at zero for 24+ hours post-deployment. Root cause: CC-backend agents lack native tool access to memory tools; the CLI bridge is high-friction. Passive channels (TOC injection, router injection) carried the load.
- **Project mis-tagging**: Formation V3's conservative prompt ("Do NOT invent new identifiers") caused entries to collapse into a single project. Fixed by encouraging organic project creation and removing the `_active_project` null fallback.
- **P&E overhead**: The assessor consumes up to 65% of total tokens. For simple tasks, standard mode with thinking enabled matches P&E accuracy at 2.7x lower wall time.
- **Agent coordination**: Ack-to-ack loops between agents in channels required explicit suppression rules. Agents defaulted to acknowledging each other's messages, creating unbounded ping-pong chains.
- **Standing digest vs. TOC**: The episodic memory TOC (always-on entry summaries) scales poorly past several hundred entries. The curation-maintained standing digest (Section 11.8) resolved this by replacing the entry-level TOC with a single coherent narrative, trading per-entry granularity for holistic context while retaining hybrid search and exact memory handles underneath it.
- **Harness-backend router dispatch**: When the router itself runs on a harness backend (Claude Code, Codex), the native tool-call dispatch path is unavailable. The unified dispatch contract (`<dispatch_worker>` XML blocks, stripped router tools, single outer iteration) emerged from repeated failures where harness-backend routers attempted to call tools they could not execute.